
基于深度强化学习的云 workflow 调度优化研究

(申请清华大学工程管理硕士专业学位论文)

培 养 单 位： 自动化系

专 业 领 域： 工程管理

申 请 人： 洪 坤

指 导 教 师： 范 玉 顺 教 授

联合指导教师： 郭 东 栋 高 工

二〇二五年五月

基于深度强化学习的云 workflow 调度优化研究

学位论文公开评阅人和答辩委员会名单

公开评阅人名单

刘 XX	教授	清华大学
陈 XX	副教授	XXXX 大学
杨 XX	研究员	中国 XXXX 科学院 XXXXXXXX 研究所

答辩委员会名单

主席	赵 XX	教授	清华大学
委员	刘 XX	教授	清华大学
	杨 XX	研究员	中国 XXXX 科学院 XXXXXXX 研究所
	黄 XX	教授	XXXX 大学
	周 XX	副教授	XXXX 大学
秘书	吴 XX	助理研究员	清华大学

关于学位论文使用授权的说明

本人完全了解清华大学有关保留、使用学位论文的规定，即：

清华大学拥有在著作权法规定范围内学位论文的使用权，其中包括：（1）已获学位的研究生必须按学校规定提交学位论文，学校可以采用影印、缩印或其他复制手段保存研究生上交的学位论文；（2）为教学和科研目的，学校可以将公开的学位论文作为资料在图书馆、资料室等场所供校内师生阅读，或在校园网上供校内师生浏览部分内容；（3）按照上级教育主管部门督导、抽查等要求，报送相应的学位论文。

本人保证遵守上述规定。

作者	_____	导师	_____
签名：		签名：	
日	_____	日	_____
期：		期：	

摘要

随着云计算与大数据技术的迅猛发展，保险公司等金融企业面临海量数据处理与复杂业务工作流的调度挑战。传统调度策略难以在异构云环境中同时兼顾任务完成时间、资源利用率与系统稳定性。本文以中国人寿基于 Apache DolphinScheduler 构建的企业级调度平台为研究场景，提出并实现了一套“特征工程增强的改进双重深度 Q 网络”（FE-IDDQN）智能调度算法。

论文首先系统梳理了工作流、深度强化学习及云计算调度的相关理论与研究现状，指出现有方法在特征建模深度、算法针对性改进及生产环境落地验证方面存在的不足。随后，围绕真实运行数据设计了面向任务级、工作流级、资源级与时序级的多维度特征工程，构建了以“缩短工作流总完成时间（Makespan）并最大化 CPU/内存利用率”为核心的多目标强化学习环境。

在此基础上，针对 DDQN 进行四项关键创新：引入注意力机制与双流网络结构，强化任务-资源匹配关系表征；设计调度关键路径感知的优先经验回放机制，提升样本利用效率；提出基于状态不确定性的自适应 ϵ -贪心探索策略，加速收敛；构建可插拔的线上推理服务，实现与 DolphinScheduler 的无缝集成。

实验部分在模拟环境和准生产环境分别展开：模拟实验表明，FE-IDDQN 相比 FIFO、HEFT、GA、PSO、DQN、DDQN 等 9 类基准算法，在 10-500+ 任务规模的多样化工作流下，Makespan 平均降低 26.8%-38.9%，CPU 与内存利用率平均提升 15-20 个百分点；在中国人寿监管报送与反洗钱两大真实业务场景中，4 周准生产验证结果显示，FE-IDDQN 将平均执行时间缩短 22%，资源利用率提升 17%，并显著改善负载均衡。

论文最后总结了算法的优势、局限及未来在图神经网络、多智能体强化学习、在线持续学习与可解释性方向的拓展前景。研究成果为金融企业复杂工作流调度提供了可落地的智能化解决方案，对提升业务时效性、资源效率与系统稳定性具有重要实践价值。

关键词：云工作流调度；深度强化学习；特征工程；双重深度 Q 网络 (DDQN)

Abstract

With the rapid evolution of cloud computing and big-data technologies, financial enterprises such as insurance companies are confronted with unprecedented challenges in scheduling massive data-processing workflows across heterogeneous cloud environments. Traditional heuristics fail to simultaneously minimize makespan, maximize resource utilization, and maintain system stability. Focusing on China Life’s enterprise-grade Apache DolphinScheduler-based platform, this thesis proposes and implements a Feature-Enhanced Improved Double Deep Q-Network (FE-IDDQN) algorithm for intelligent workflow scheduling.

After reviewing related theories and identifying gaps in existing work—particularly insufficient feature modeling, limited algorithmic tailoring, and lack of production-grade validation—we first construct a comprehensive multi-dimensional feature engineering pipeline. The pipeline distills static and dynamic characteristics at the task, workflow, resource, and temporal levels from 91 million real-world task instances, and embeds them into a multi-objective reinforcement-learning environment whose primary goals are to shorten overall workflow completion time (makespan) and maximize CPU/memory utilization.

Four key innovations are introduced to the standard DDQN: an attention-augmented dual-stream architecture that captures task-resource affinities; a critical-path-aware prioritized experience replay that improves sample efficiency; a context-adaptive ϵ -greedy exploration strategy that accelerates convergence; a pluggable online inference micro-service enabling seamless integration with DolphinScheduler.

Extensive experiments are conducted in both simulated and quasi-production environments. Simulation results on synthetic and real-trace-driven datasets (10–500+ tasks) show that FE-IDDQN outperforms nine baselines—including FIFO, HEFT, GA, PSO, DQN, and DDQN—reducing makespan by 26.8 %–38.9 % and increasing CPU and memory utilization by 15–20 percentage points. A four-week quasi-production trial on China Life’s regulatory-reporting and anti-money-laundering workflows demonstrates an average 22 % reduction in execution time and a 17 % improvement in resource utilization, while significantly balancing cluster load.

The thesis concludes with a discussion of advantages, limitations, and future directions such as graph neural networks, multi-agent reinforcement learning, continual online learning, and explainability. The findings provide a practical, intelligent scheduling solution for complex financial workflows, enhancing business timeliness, resource efficiency, and system stability.

Keywords: Cloud Workflow 1; keyword 2; keyword 3; keyword 4; keyword 5

目 录

插图清单

附表清单

符号和缩略语说明

第 1 章 绪论

1.1 研究背景

随着云计算技术的不断发展，公司在其业务数据处理中面临着越来越复杂的计算任务。在云计算环境中，调度问题是一个关键的优化问题。该问题涉及到如何合理安排云计算中的任务执行顺序，以最大化系统性能、降低资源利用成本，和带宽开销。根据本章的理论模型可以很好地测算出各种条件下及不同应用中的并满足各项约束条件。如何高效地利用云计算资源，更快速度、更省资源的调度数据成为了一项非常值得研究的任务。

在保险数字化领域，保险数据服务涉及商务智能、监管报送、客户信息推送等多个方面的综合服务。在这个领域中，处理数据的工作流任务调度具有极其重要的意义和作用。首先，数据调度可以帮助保险公司更加高效地管理和分析大量的数据，从而更好地理解市场和客户需求，为公司的决策提供有力的支持。其次，数据调度还可以帮助保险公司更好地遵守监管要求，及时准确地完成各种监管报送任务。最后，数据调度还可以帮助保险公司更好地服务客户，通过精准的客户信息推送，提高客户满意度和忠诚度。因此，数据调度在保险数据服务领域中具有非常重要的地位和作用。而数据调度工作流执行时间最小化，则可以帮助保险公司更加高效地完成数据调度任务，提高数据处理的效率和准确性，从而更好地支持公司的业务发展和决策。

在保险公司数据调度生产环境中，工作流任务的运行历史数据是非常宝贵的资源。通过利用这些历史数据，并结合机器学习算法来优化调度，可以显著提高工作效率和资源利用率，从而降低成本。这种方法不仅适用于保险业务，还可以推广到其他行业和领域，对提升企业运营效率和竞争力具有重要意义。在实际操作中，通过对历史数据的分析和建模，可以预测工作流任务的执行时间和资源需求，从而优化调度方案，使得整体的 **MAKESPAN** 最小化。这种基于历史数据和机器学习的调度优化方法，不仅能够提高任务执行的效率和准确性，还可以减少人工干预和错误，降低运营风险，提升企业的核心竞争力。因此，利用历史数据和机器学习算法来优化调度，在保险公司这样的实际生产环境中具有重要的实际意义和广阔的应用前景。

1.1.1 workflow 概念

workflow 的概念起源于生产组织和办公自动化领域，其基本思想是将复杂的工作过程拆分为定义清晰的任务和角色，并依据既定的规则和过程进行有序执行和监控。其目标在于提高办事效率、降低生产成本、优化企业的生产经营管理水平，从而增强整体竞争力。workflow 技术是支持企业业务过程重组（business process reengineering, BPR）与经营过程自动化（business process automation, BPA）的重要技术手段。

1993 年 workflow 管理联盟（workflow management coalition, WFMC）的成立标志着 workflow 技术开始进入相对成熟的阶段。workflow 管理联盟将 workflow 定义为“能够完全或者部分自动执行的业务过程，它根据一些列过程规则、文档、信息或任务能够在不同的执行者之间进行传播与执行”。在学术界，Georgakopoulos 等学者从执行层面出发，将 workflow 概括为“将一组任务组织起来完成某个业务过程”，并强调了任务的触发条件、执行顺序以及执行主体的多样性。

在国内，范玉顺^[1]教授在其著作《workflow 管理技术基础》中进一步提出，workflow 是一种反映业务流程的计算机化的模型，是为了在先进计算机环境支持下实现业务过程集成与业务过程自动化而建立的可由 workflow 管理系统执行的业务模型。与 WFMC 和 Georgakopoulos 的定义相比，范教授的表述更突出业务过程的建模与系统化集成，不仅强调了 workflow 在技术层面的执行机制，更强调其企业信息化与经营管理优化中的价值。这一定义为后续将 workflow 应用于企业流程管理、调度优化及智能化提供了坚实的理论基础。

1.1.2 深度强化学习的相关概念

深度强化学习（DRL）是一种结合了深度学习（DL）和强化学习（RL）的技术。深度学习（Deep Learning, DL）是一种利用深层神经网络从大量数据中自动学习特征的机器学习方法。它特别适合于处理高维输入数据，如图像、声音和文本。强化学习（Reinforcement Learning, RL）是一种机器学习方法，它通过智能体（agent）与环境的交互来学习如何做出最优决策。智能体根据环境的状态采取行动，并从环境中获得奖励或惩罚，通过这种方式不断调整其行为策略。

深度强化学习算法根据具体是否需要环境模型可以分为模型基础（model-based）和模型无关（model-free）两大类。模型无关的深度强化学习算法不直接使用或学习环境的模型。它们主要通过与环境交互来学习策略或价值函数。典型的模型无关算法包括 DQN（Deep Q-Network）、DDQN（Double Deep Q-Network）、A3C（Asynchronous Advantage Actor-Critic）等。这些算法直接从经

验中学习，不需要对环境动态进行显式建模。模型基础的深度强化学习算法试图学习或使用环境的模型来预测状态转移和奖励。这些算法通常会构建一个环境的近似模型，并使用这个模型来进行规划和决策。

1.1.3 云计算的基本概念

云计算（Cloud Computing）是分布式计算的一种，指的是通过共有网络或者私有网络将数据计算处理程序分解成多个子程序，然后通过多个服务器组成的系统进行处理和分析这些小程序得到结果并返回给用户。随着近年来互联网和移动互联网的飞速发展，数据量越来越大，云计算就是在数据量越来越大、计算任务越来越繁重的基础上提出来的。

云计算作为还在发展中的概念，尚未有全球统一的标准分类。但是根据业界共识，可以从不同角度对其进行分类。从部署区域或者可用范围的角度来分，可以把云计算类型分为私有云（Private Cloud）、公有云（Public Cloud）、社区云（Community Cloud）和混合云（Hybrid Cloud）四种类型。

按照服务类型，云计算可以分为基础设施即服务（Infrastructure as a service，简称 IAAS）、平台即服务（Platform as a service，简称 PAAS）、软件即服务（Software as a Service，简称 SAAS）、数据即服务（Data as a service，DAAS）四类。

1.2 国内外研究现状

云 workflow 调度问题因其复杂性和重要性，一直是学术界和工业界的研究热点。国内外学者和研究机构已从不同角度提出了大量的调度算法和优化策略。这些研究大致可以分为传统启发式与元启发式算法、以及近年来兴起的基于机器学习（特别是深度强化学习）的调度算法两大类。

1.2.2 传统启发式与元启发式算法回顾

传统的启发式算法通常基于一组预定义的规则来做出调度决策，它们实现简单、计算开销小，在特定场景下能够获得较好的效果。先来先服务（First-Come, First-Served, FCFS），按照任务到达的顺序进行调度，简单公平但未考虑任务特性和资源状态，容易导致短任务等待长任务的问题。最短作业优先（Shortest Job First, SJF），优先调度估计执行时间最短的任务，能够有效降低平均等待时间，但可能导致长任务饥饿。最小完成时间优先（Earliest Finish Time, EFT），将任务分配给能使其最早完成的可用资源，是一种贪心策略。关键路径算法（Critical Path

based algorithms), 如 HEFT (Heterogeneous Earliest Finish Time) 和 CPOP (Critical Path on a Processor), 它们首先计算任务在 DAG 图中的关键路径, 并优先调度关键路径上的任务, 旨在最小化整体完成时间 (Makespan)。HEFT 是异构环境下广泛使用的基准算法之一。

元启发式算法, 如遗传算法 (Genetic Algorithm, GA)、蚁群算法^[2] (Ant Colony Optimization, ACO)、粒子群优化^[3, 4] (Particle Swarm Optimization, PSO)、模拟退火 (Simulated Annealing, SA) 等, 通过模拟自然进化或物理过程来搜索近似最优解。这些算法通常具有较强的全局搜索能力, 能够处理复杂的约束条件和优化目标, 在云 workflow 调度中也取得了一定的成果。例如, 文献^[5]使用遗传算法对标准 jobshop 问题的调度序列进行编码和进化, 宋寒冰^[6]将改进的粒子群算法 (IPSO 算法) 与云计算任务调度结合, 主要关注云用户任务与资源之间的匹配问题, 建立了数学模型和物理模型, 邱模奇^[7]使用改进的蚁群算法 R_ACO 优化云 workflow 调度问题, 他使用有向无环图定义 workflow 过程模型和资源模型。然而, 元启发式算法往往存在收敛速度慢、参数设置敏感、易陷入局部最优等问题, 且对于大规模动态变化的调度环境, 其适应性和实时性面临挑战。黄志刚^[8]等人利用 HEFT (Heterogeneous Earliest Finish Time) 算法, 针对任务运行在各个虚拟机上的最小完成时间 MET (Minimum Execution Time) 进行优化, 合并 workflow 中的管道任务从而降低了任务部署在同一台虚机上的数据传输时间。

1.2.2 基于深度强化学习的调度算法进展

随着人工智能技术的发展, 基于机器学习的调度方法受到了越来越多的关注。其中, 深度强化学习 (Deep Reinforcement Learning, DRL) 结合了深度学习的感知能力和强化学习的决策能力, 展现出解决复杂序列决策问题的巨大潜力, 非常适合应对云 workflow 调度的动态性和不确定性。

DRL 通过让智能体 (Agent) 与环境 (云调度系统) 交互, 根据获得的奖励或惩罚来学习最优的调度策略。智能体观察环境状态 (如任务队列、资源负载), 执行调度动作 (如将任务分配给特定资源), 并从环境中获得反馈 (如任务的完成时间、资源利用率的变化等)。常见的算法分为以下三类:

基于值函数的方法: 深度 Q 网络 (Deep Q-Network, DQN) 及其变种是 DRL 中应用最广泛的算法之一。DQN 使用深度神经网络来近似最优动作值函数 $Q^*(s,a)$ 。通过经验回放 (Experience Replay) 和目标网络 (Target Network) 等技术, DQN 能够稳定地学习高维状态空间下的策略。双重深度 Q 网络 (Double DQN, DDQN^[9]) 通过解耦目标 Q 值计算中的动作选择和动作评估, 有效缓解了 DQN 中的

过高估计问题，通常能获得更好的性能和更稳定的学习过程。许多研究已尝试将 DQN 或 DDQN 应用于云任务调度，通过定义合适的状态、动作和奖励函数，来优化调度目标如 Makespan、成本等。

近年来，将 DRL 应用于云 workflow 调度的研究层出不穷。例如，有研究者利用 DQN 来决定任务的分配顺序和资源选择；有工作^[10]通过 A3C 来优化多目标（如时间和能耗）的调度策略；还有研究探索使用图神经网络（Graph Neural Network, GNN）来处理工作流的 DAG 结构，并结合 DRL 进行调度决策。这些研究表明，DRL 在处理动态环境、学习复杂策略方面具有显著优势。

郭嘉伟^[11]针对基于 DRL 的分布式数据处理系统调度策略所面临的挑战，提出了针对性的解决策略，涉及训练框架的实施、特征学习能力提升以及并行训练加速三个维度。经过对所提方案进行详尽的评估，研究证实了这些策略在不同维度上均展现出卓越的性能表现。

文献^[12]指出传统的任务调度方法在动态变化的云环境下难以适应，而深度强化学习方法因其在决策问题上的优势，成为解决复杂云环境下任务调度问题的有效途径。此文献提出了一种基于双深度 Q 网络（Double DQN）的任务调度算法（SDDQN），通过建模状态空间、动作空间和奖励函数，优化任务成功率、平均响应时间和执行成本。在模拟数据和真实世界工作负载上验证了 SDDQN 算法的性能，结果表明该算法在减少任务平均响应时间和执行成本方面表现良好，同时保持了较高的任务成功率。开发了一个任务调度系统，该系统参考了 CloudSim Plus 仿真平台的设计思路，使用 Python 语言实现了调度算法，并提供了用户界面以可视化展示调度结果。

文献^[13]提出 RLSD 算法提高任务调度的效率和资源利用率。RLSD 算法的设计包括三个主要部分：调度器、智能体决策与训练组件、指标度量值采集器与接收器。算法通过定义状态（state）、动作（action）和奖励（reward）来构建模型，并使用深度前馈神经网络来计算 Q 值，从而实现智能体的自主学习和决策。

第 1 章 绪论

表 1 云计算任务调度算法研究文献总结

类型	算法描述	特点
文档 [14]元启发式	半离线阶段: PSO+SFO。在线阶段: 根据新问题的参数, 使用规则集确定场景类别	优点: 在线阶段资源利用率高。缺点: 扩展性较弱
文档 [15]元启发式	提出了基于 QPSO 的任务调度算法, 通过量子粒子的运动模型进行全局搜索	优点: QPSO 算法具有良好的全局搜索能力; 缺点: QPSO 算法的迭代时间较长, 时间开销较大
文档 [16]混合式	PDTS: 按任务图的拓扑顺序排序子任务, 优先为空闲速度最快的工人分配可执行任务 DETS: 设计编码、解码和初始化方法解决任务调度问题	PDTS: 简单高效, 具有明确的理论近似比保证; DETS: 搜索性能强; CWR: 有效平衡探索和利用, 通过调整项学习工人的计算能力
文档 [17]深度强化学习	每个智能体基于局部网络状态信息生成关系图, 利用 RGCN 提取特征, 并根据 SAC 框架进行训练和执行。	优点: 可以快速适应动态变化的网络环境, 提高任务执行效率, 同时实现节点和网络负载均衡。缺点: 计算开销较大, 需要大量样本进行训练
文档 [18]深度强化学习	D3QN 算法构建状态空间, 设计状态感知算法, 设计时间步自适应机制, 设计复合奖励机制	优点: 在满足连续性约束下, 能够优化高维目标。缺点: 动态云计算服务环境下的学习效果不佳
文档 [19]深度强化学习	使用布谷鸟搜索算法(ICSA)进行资源分配, 深度强化学习(DRL)用于资源管理	优点: 提高了资源利用率, 减少了执行时间, 增强了安全性。算法缺点: 算法复杂度高, 计算量大
文档 [20]深度强化学习	提出了自适应多目标深度 Q 网络 (AMDQN), 一种基于分层深度强化学习的算法, 包含控制器 DQN 和执行器 DQN	优点: 采用分层结构实现多目标优化, 减少调度规则选择的复杂度。
文档 [21]深度强化学习	采用了基于规则的深度强化学习算法, 使用值函数网络和策略网络进行决策	优点具有很好的泛化能力、可扩展性和鲁棒性; 缺点: 训练速度较慢, 需要大量样本

1.2.3 现有研究在 DDQN 应用、特征工程及实际落地方面的不足

尽管基于 DRL 的云 workflow 调度取得了显著进展，但在 DDQN 的应用深度、特征工程的系统性以及研究成果在实际生产环境落地方面，仍然存在一些改进的空间，主要体现在以下几个方面。

DDQN 算法的针对性改进不足。DDQN 相较于 DQN 在稳定性和收敛性上有所改进，但是直接将其应用于复杂的云 workflow 调度场景时，依旧面临状态空间巨大、动作空间复杂、收敛速度慢等问题。现有研究普遍缺乏结合 workflow 调度特点的针对性改进，例如，如何设计更有效的网络结构来捕捉任务依赖和资源关联，如何改进经验回放以提高样本利用率，以及如何改进探索策略以加快收敛。这些问题亟需要更具针对性的研究。

特征工程的系统性和深度有待加强。强化学习的性能高度依赖于状态标识的质量，而有效的特征工程是构建高质量状态表示的关键。然而，现有研究对任务历史执行特性与资源需求模式、DAG 拓扑信息与关键路径特征、云环境节点负载与网络状况等多维信息的提取、融合与表示尚不够深入。尤其在如何利用大规模运行日志与历史调度数据自动学习关键特征方面，研究尚处于初步阶段，难以充分支撑高效的调度决策。

与真实调度系统的集成和验证不足。目前大多数研究主要依赖 CloudSim 等仿真平台或标准 workflow 数据集进行实验验证。这些仿真环境虽然能够说明算法的理论可行性，但与真实的企业级调度系统（如 Apache DolphinScheduler）在规模、复杂性和运行环境上存在显著差距。缺乏在生产环境中的集成与验证，限制了 DRL 调度算法在工程实践中的推广和价值体现。

多目标优化与动态适应性的平衡。云 workflow 调度往往涉及任务完成时间、资源利用率、能耗等多目标优化问题，同时还需应对环境动态变化。在现有 DRL 研究中，如何在策略训练中有效平衡多个可能冲突的优化目标，并确保算法能够快速适应突发变化，仍是尚未解决的难点。

综上所述，现有研究在算法针对性、特征建模深度、实际落地验证及动态适应性等方面均存在不足。本研究拟在 DDQN 的结构改进、深度特征工程方法设计以及与中国人寿 DolphinScheduler 平台的实际融合应用上进行探索与突破，以期提出一种更高效且可落地的智能 workflow 调度优化方案。

1.3 研究目标与主要内容

1.3.1 具体研究目标

本研究通过结合先进的深度强化学习技术与精细化的特征工程方法，针对以 `apache dolphinscheduler` 为代表的企业级云 workflow 调度平台，开发一套能够缩短 workflow 平均执行时间并提升计算资源利用率的智能调度优化方案。具体研究目标如下。

设计并实现一套面向云 workflow 调度的有效特征工程方案。从 `dolphinscheduler` 等平台的实际运行数据中，系统性的提取和构建能够全面反映任务特性、workflow 结构以及资源环境状态的多维度特征，为强化学习智能体提高高质量、信息丰富的状态输入。

提出一种改进的 DDQN 调度算法。针对云 workflow 调度的特点和挑战，对现有的 DDQN 算法进行创新性改进。引入注意力机制增强对关键信息的捕捉，对网络结构进行优化，设计更符合调度任务特性的优先经验回放策略等，以提升算法的学习效率、收敛速度和最终性能。

构建一个以缩短执行时间和提升资源利用率为核心的多目标优化调度模型。将 workflow 执行时间最小化和 CPU/内存资源利用最大化作为主要的优化目标，设计相应的优化学习环境，使智能体能够学习到平衡这两个目标的调度策略。

通过仿真实验和实际案例验证所提方法的有效性。在模拟环境和实际生产环境中的真实应用场景中，对所提的特征工程方法和改进 DDQN 算法进行全面的实验评估。通过与传统调度算法、标准 DDQN/DQN 算法进行对比，量化分析其在缩短执行时间、提升资源利用率等方面的性能优化。

1.3.2 论文核心研究内容

1、数据驱动的特征工程与环境建模

深入分析 `dolphinscheduler` 的历史运行数据，理解数据特性和潜在价值。设计并实施一套完整的特征工程流程，包括数据清洗、预处理、多维度特征（任务级、workflow 级、资源级、时间序列特征等）的提取与选择。在这些提取的特征基础上，构建强化学习智能体的状态空间、动作空间，并设计能够反映多目标优化的奖励函数，从而建立一个贴近实际的云 workflow 调度强化学习环境。

2、基于特征增强的改进 DDQN 算法设计与实现

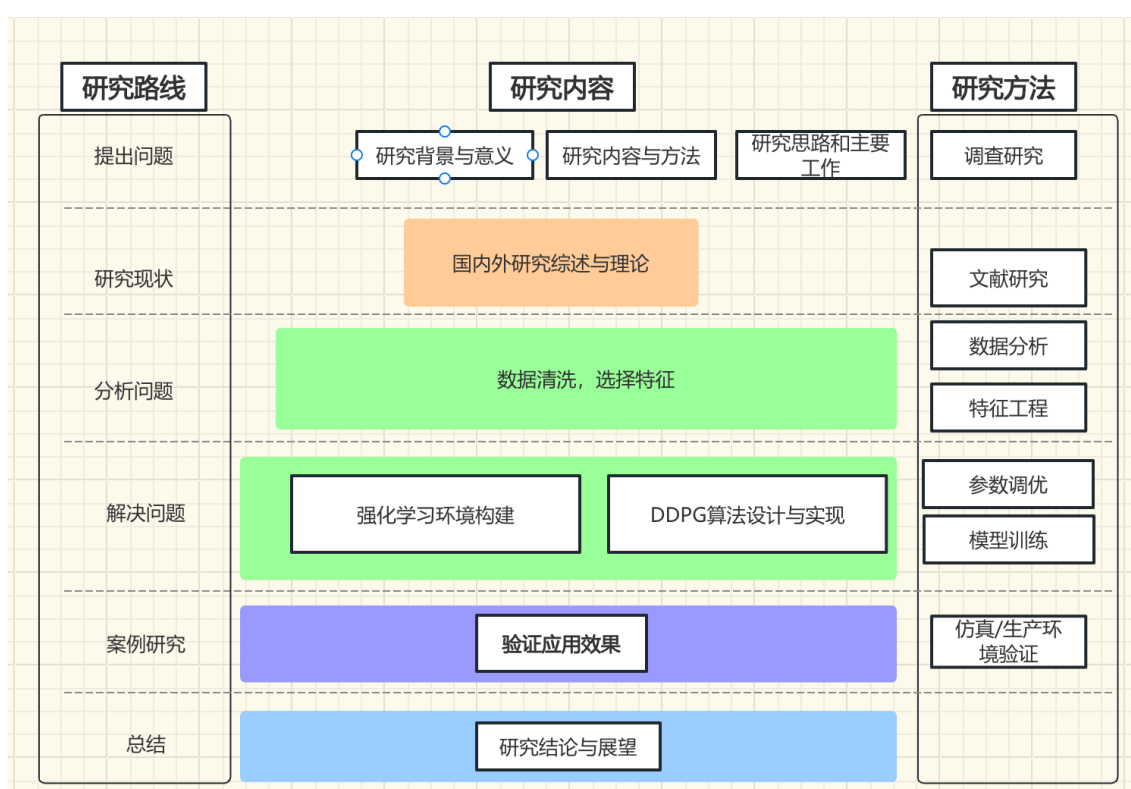
详细阐述所提出的改进 DDQN 算法的原理、架构和关键创新点，讨论算法的训练流程、参数设置、收敛性分析以及复杂性分析，强调特征工程如何增强 DDQN 算法对环境状态的感知能力，从而提升其决策质量。

3、评估实验结果与实际应用验证

设计详细的实验方案，包括实验平台选择、数据集构建、对比算法选择以及性能评价指标定义。通过大量的仿真实验，系统评估特征工程的有效性以及改进 DDQN 算法相较于基线算法的性能提升。结合中国人寿 DolphinScheduler 平台的实际应用背景，选取典型业务场景进行案例研究，验证所提方法在真实环境下的应用效果和可落地性，并讨论实践中可能遇到的挑战与解决方案。

1.4 技术路线图

本文的技术路线如下图所示



1.5 论文的组织结构

本论文共分为七章，其组织结构如下：

第一章：绪论。主要介绍研究背景与意义，阐述云 workflow 调度的挑战、保险行业大数据处理对调度系统的新要求，并以 `DolphinScheduler` 为例探讨企业级调度平台的优化需求。明确本研究以缩短执行时间与提升资源利用率为核心目标，综述国内外相关研究现状，分析现有研究的不足，最后给出本论文的研究目标、主要内容、技术路线图和整体组织结构。

第二章：相关研究工作。详细介绍模型、关键目标与评价指标；介绍 `Apache DolphinScheduler` 的核心机制、系统架构及用户实践环境；系统阐述深度强化学习的基本原理，包括马尔可夫决策过程、Q-Learning、DQN 及 DDQN 算法；最后介绍特征工程的基本概念、方法及其在调度优化中的应用潜力。

第三章：面向云 workflow 调度的特征工程与强化学习环境设计。首先对云 workflow 调度问题进行形式化定义和数学建模，构建多目标优化函数。随后，围绕 `DolphinScheduler` 运行数据展开特征工程设计，涵盖数据来源与预处理流程，以及任务级、workflow 级和资源环境级等多维特征的提取、选择与表示方法。最后，设计强化学习环境的关键要素，包括状态空间、动作空间与奖励函数的构建方案。

第四章：提出特征增强的改进型 DDQN 算法（FE-DDQN）的总体框架，重点阐述特征增强机制、优化后的网络结构与经验回放策略，以及探索与利用的平衡方法。进一步给出算法的训练流程、参数设置、复杂度分析与核心伪代码。

第五章：实验与结果分析。介绍实验环境搭建和数据准备工作，包括处理后的 `DolphinScheduler` 生产数据子集及标准 workflow 数据集。选择启发式算法与经典强化学习算法作为对比基准，通过大量仿真实验评估特征工程对调度性能的作用，并在多维度、多场景下比较 FE-DDQN 与其他算法的性能表现。最后，总结其性能优势，分析关键提升因素，同时指出算法的局限性及在实际应用中的意义。

第六章：在中国人寿的数据调度平台上进行验证。首先介绍中国人寿云调度平台的整体概况与应用背景，随后描述 FE-DDQN 算法在该平台的集成实现过程。通过典型业务案例（如监管报送 workflow 优化与反洗钱数据分析 workflow 优化），对算法在真实场景下的性能进行验证，并进一步评估其长期运行效果，总结实施方案与实践经验。

第七章：总结与展望。全面总结本论文的研究工作和主要创新成果，客观分析研究存在的局限性，并对未来值得进一步研究的方向进行展望。

第 2 章 相关研究工作

本章将系统介绍与云 workflow 智能调度相关的国内外研究进展。首先，概述云 workflow 调度的问题特性与评价指标，并介绍本研究的实践背景平台——Apache DolphinScheduler（2.1 节）。其次，论证特征工程在基于深度强化学习的调度系统中的必要性与定位（2.2 节）。最后，系统性地回顾深度强化学习在云 workflow 调度中的研究现状，明确现有研究的不足，阐明本文的研究定位（2.3 节）。

2.1 云 workflow 调度概述

2.1.1 云计算环境下的调度特性

将 workflow 部署到云计算环境中执行，即形成云 workflow。云计算环境在带来按需获取资源、弹性伸缩、高可用性等优势的同时，也给 workflow 调度带来了新的特性和挑战。Zhou 等人^[22]在其关于深度强化学习云计算资源调度的综述中系统性地归纳了这些特性，Hou 等人^[23]也从能效优化的角度进行了专门分析。这些特性主要包括：（1）资源异构性——云平台提供多种类型和配置的计算资源，在 CPU、内存、存储、网络等方面存在显著差异，Chen 等人^[18]指出异构资源的合理利用是提升调度性能的关键；2）资源动态性——资源状态因多租户共享、按需租用释放和潜在故障而动态变化，Lu 等人^[24]发现资源状态的不确定性对调度决策质量有显著影响；（3）弹性伸缩——调度系统可根据负载动态调整资源规模以平衡性能和成本；（4）按需付费——调度决策需同时考虑性能和经济指标，Cheng 等人^[25, 26]从抢占式调度和实例选择角度证明了成本感知调度的重要性；（5）虚拟化与多租户——共享资源可能导致性能干扰；（6）数据局部性——将计算调度到靠近数据的节点可显著减少传输开销；（7）边缘-云协同——Wang 等人^[27]和 Zhou 等人^[28]分别研究了边缘-雾计算和万物互联场景下的 DRL 调度问题，多维约束进一步增加了调度复杂性。

这些特性使得云 workflow 调度比传统环境下的调度更为复杂，对调度算法的智能化、自适应性和鲁棒性提出了更高要求。本研究所面对的正是这样一个具有资源异构、负载动态变化和 DAG 依赖约束的复杂调度环境

2.1.2 关键调度目标和性能评价指标

云 workflow 调度的目标是将 workflow 中的任务有效地映射到可用的云资源上执行，本研究关注的核心目标是缩短执行时间和提升资源利用率。常见的关键调度目标和性能评价指标包括：**Makespan** (总完成时间，衡量调度性能最常用和最重要的指标)、资源利用率 (CPU/内存等有效利用的程度)、执行成本 (按需付费模式下的经济指标)、吞吐量 (指单位时间内系统能够完成的 workflow 数量或任务数量)、任务平均等待时间与周转时间、负载均衡度、可靠性/成功率等。

不同的应用场景对评价指标有不同的侧重。例如，Jayanetti 等人^[29]将能耗和时间作为联合优化目标；Sellami 等人^[30]以能量感知为核心评价维度；Ibrahim 和 Askar^[31]同时优化完成时间、资源利用率和能耗三个指标。这些研究表明，构建灵活的多目标评价体系对于评估云 workflow 调度算法的综合性能至关重要。本研究将重点优化 **Makespan** 和 CPU/内存利用率，奖励函数的设计和实验评估都将围绕这两个核心指标展开。

2.1.3 Apache DolphinScheduler 调度平台

Apache DolphinScheduler 是一款开源的、分布式的可视化 DAG workflow 任务调度平台，致力于解决大数据任务之间复杂的依赖关系。Pawar 等人^[32]在对 workflow 引擎的分类学研究中指出，现代 workflow 引擎在任务自动化中扮演着越来越重要的角色，而 DolphinScheduler 凭借其企业级特性——高可用性、高可靠性、丰富的任务类型支持以及良好的社区生态，在众多企业中得到了广泛应用，特别是在 ETL、数据仓库、机器学习平台等场景。

DolphinScheduler 的核心是基于 DAG (有向无环图) 描述 workflow 中任务之间的依赖关系，一个任务只有在其所有前驱任务都成功完成后才能被调度执行。Koslovski 等人^[32]在其 DAG workflow 调度研究中指出，DAG 结构是描述任务间依赖关系的最自然且高效的形式。DolphinScheduler 采用分布式 Master-Worker 多主多从架构：MasterServer 负责 DAG 任务切分、任务分发和状态监控，采用无中心化设计以实现高可用；WorkerServer 负责具体任务的执行，可部署于不同物理机或虚拟机上形成计算集群；ZooKeeper 提供集群管理、Master 选举和分布式锁等协调服务；此外还包括元数据库 (存储 workflow 定义、任务实例状态等)、RESTful API 接口层和可视化 Web UI。

这种架构使得 DolphinScheduler 能够水平扩展以应对不断增长的业务需求。作为本研究的实践验证平台，DolphinScheduler 的 DAG 调度模型、丰富的元数据和运行日志为构建深度强化学习调度智能体提供了理想的数据基础和应用场景。

2.2 深度强化学习在调度优化中的研究现状

随着云计算技术的快速发展和 workflow 应用的日益复杂化,传统的启发式调度算法在处理动态、异构的云环境时面临诸多挑战。近五年来,深度强化学习(DRL)因其在处理高维状态空间和复杂决策问题上的优势,逐渐成为云 workflow 调度领域的研究热点。Zhou 等人^[22]在其发表于 *Artificial Intelligence Review* 的综述中系统性地回顾了 DRL 在云计算资源调度中的方法论进展,Hou 等人^[23]则从能效优化角度对基于 DRL 的云环境任务调度进行了专题综述。这些综述性工作表明,DRL 已从实验性探索阶段逐步走向更加成熟的应用阶段。本节将从基于 DRL 的云 workflow 调度研究、DDQN 算法的改进、基于特征工程的 DRL 调度研究三个维度进行系统性回顾,并在此基础上分析现有研究的不足与本文的研究定位。。

2.2.1 基于 DRL 的云 workflow 调度研究

根据 Gu 等人^[33](2025)的综述性研究,将 DRL 应用于云 workflow 调度通常遵循"环境建模—智能体设计—训练与部署"的技术框架。环境建模是指将云 workflow 调度系统抽象为强化学习环境,包括状态空间、动作空间、奖励函数和状态转移。智能体设计是指根据调度问题的特点选择和设计适合的 DRL 算法,如 DQN、DDQN、A3C 等。Zhou 等人^[22]进一步将 DRL 云调度方法分为基于值函数的方法(如 DQN、DDQN)、基于策略梯度的方法(如 A2C、PPO)和演员-评论家方法(如 A3C、SAC),并分析了各类方法在不同调度场景下的适用性和优劣。

近五年来,基于 DRL 的云 workflow 调度研究主要集中在以下几个方向:

(1) 多目标优化。云 workflow 调度通常需要同时考虑多个目标,如执行时间、资源利用率、能耗、成本等。Mangalampalli^[34]等人(2024)提出了基于改进异步优势演员-评论家算法(IA3C)的自适应任务调度器,通过综合考虑完成时间、资源成本和能耗三个目标,在多云环境中实现了显著的性能提升。同一研究团队的 Mangalampalli 等人^[35]还提出了 DRLBTSA (Deep Reinforcement Learning Based Task-Scheduling Algorithm) 算法,面向异构云环境中的多媒体任务调度,通过 DRL 智能体根据当前到达任务和虚拟资源运行状态自动做出调度决策,在 Makespan、SLA 合规率和能耗三个指标上均优于传统启发式方法。文献^[36]提出了 DTSS-DQN 算法,通过引入五种复合调度规则,根据不同时间段内到达的任务数量和虚拟机状态选择最合适的调度规则,显著降低了平均响应时间并提高了资源利用率。文献^[37]针对多用户、多队列、多数据中心的复杂环境,提出了以优化传

输时间、等待时间和执行时间为目标的 DRL 调度方法。Ibrahim 和 Askar^[31]

(2023) 提出了基于多目标 DRL 的雾计算智能调度策略, 同时优化任务完成时间、资源利用率和能量消耗。Cheng 等人^[38] (2024) 则通过设计综合性奖励函数实现了执行时间和资源利用率的联合优化。上述研究表明, 多目标 DRL 调度已经形成了较为完善的方法体系, 但如何在目标间进行动态权衡仍是开放问题。

(2) 成本感知调度。在按需付费的云计算环境中, 执行成本是用户高度关注的指标。Cheng 等人^[25] (2024) 在 *IEEE Transactions on Sustainable Computing* 上提出了基于 DRL 的抢占式成本感知云作业调度方法, 创新性地将作业抢占机制引入 DRL 调度框架, 在训练过程中同时优化作业到虚拟机的分配和虚拟机内部的队列排序, 在保证用户期望响应时间的前提下显著降低了虚拟机执行成本。Cheng 等人^[26] (2022) 早期的研究探索了使用 DRL 进行成本感知的云实例作业调度, 通过学习不同实例类型的价格-性能比来优化资源选择。Zhang 等人^[39]

(2023) 提出了结合遗传算法和 DRL 的成本感知实时 workflow 调度系统, 将 GA 的全局搜索能力与 DRL 的在线决策能力相融合, 在满足截止期限约束的同时降低了 15-25% 的执行成本。成本感知调度的研究趋势表明, 单纯的性能优化不足以满足实际云环境的需求, 将经济指标纳入优化目标是 DRL 调度走向实用化的重要方向。

(3) 边缘-云协同调度。随着边缘计算和雾计算的兴起, DRL 在边缘-云协同调度中的应用成为新的研究热点。Wang 等人^[27] (2024) 在 *Future Generation Computer Systems* 上提出了 DRLIS (Deep Reinforcement Learning-based IoT application Scheduling) 算法, 面向异构边缘-雾计算环境中 DAG 结构的物联网应用调度, 在 FogBus2 无服务器框架中实现了实际部署, 相比元启发式和其他 RL 方法分别降低了 55% 的负载不均衡度和 37% 的响应时间。Jayanetti 等人^[29]

(2022) 针对边缘-云计算环境中带优先约束的任务提出了基于 DRL 的能量和时间联合优化调度方法。Zhou 等人^[28] (2022) 提出了面向万物互联的边缘使能两阶段调度方法, 采用 DRL 在边缘侧进行任务分配和在云端进行资源优化的两阶段策略。Zheng 等人^[40] (2022) 研究了基于 DRL 的边缘计算工作负载调度, 通过设计适应边缘场景的状态空间和动作空间实现了计算任务在边缘节点间的智能分配。Lu 等人^[24] (2024) 在 *IEEE Internet of Things Journal* 上提出了 A2C-DRL 算法, 构建了服务器资源调度系统 (SRSS) 模型, 引入资源约束模块和负载均衡因子, 在真实数据集上将任务拒绝率降至 0.1% 以下, 负载均衡因子比 DDQN、REINFORCE 等 7 种算法低 0.19-0.443。Sellami 等人^[30] (2022) 则在 SDN 使能的物联网网络中利用 DRL 实现了能量感知的任务调度和卸载。边缘-云协同场景的

核心挑战在于异构环境下的决策空间急剧扩大，这进一步凸显了高效状态表示（即特征工程）的重要性。

（4）DAG 工作流专用调度与动态环境处理。工作流的 DAG 结构是区别于一般任务调度的关键特征。Koslovski 等人^[41]（2024）在 *Future Generation Computer Systems* 上专门研究了基于 Actor-Critic 深度强化学习的 DAG 工作流调度方法，创新性地提出了元调度器（meta-scheduler）架构——不是设计新的调度策略，而是利用 AC 强化学习智能体根据 DAG 队列结构和数据中心资源状态动态选择最优的已有调度策略组合（包括队列排序策略和服务器选择策略），在 bounded slowdown 和接受率指标上超越了固定策略调度器。在动态环境处理方面，Sun 和 Mu^[42]（2020）的研究指出，多智能体深度强化学习在处理动态、非静态环境方面具有显著优势。Sun 等人^[43]（2025）提出的基于 DDQN 的调度算法通过优化网络结构和训练策略，显著提高了大规模任务场景下的性能和效率。文献^[44]将 LSTM 与 DQN 结合成 L-DQN 算法，解决了传统 Q 学习在高维状态空间中的局限性。Fan 等人^[45]（2022）提出了 DRAS 算法，针对高性能计算集群调度问题，在处理数千并发作业时仍能保持高效的调度决策。这些研究共同表明，处理 DAG 依赖结构和动态环境变化是 DRL 调度算法走向实际应用的核心技术难点。

2.2.2 DDQN 算法的改进研究

双重深度 Q 网络（DDQN）因其在减少 Q 值过高估计、提高学习稳定性方面的优势，被广泛应用于云 workflow 调度研究中。Sun 等人^[43]（2025）的研究表明，DDQN 在任务调度中能够动态调整任务优先级和资源分配策略，相比标准 DQN 在收敛速度和最终性能上均有 10-15% 的提升。为了更好地适应云 workflow 调度的特点，研究者们对 DDQN 算法进行了多方面的改进。

（1）网络结构改进。传统 DDQN 使用的全连接神经网络在处理工作流的图结构信息时存在局限。Chen 等人^[46]（2023）提出将图神经网络（GNN）与 DDQN 结合，更好地捕捉任务间的依赖关系和工作流拓扑结构，实验表明这种结构在处理复杂 DAG 工作流时能够比传统 DDQN 减少约 20% 的平均完成时间。

Huang 等人^[47]（2024）在 *Expert Systems With Applications* 上提出了 GDRL（GNN-based DRL）方法，以拼接析取图（stitched disjunctive graph）作为分布式作业车间调度的状态表示，设计了操作-工厂对（operation-factory pair）的联合动作空间，并引入概率增强机制防止作业在单一工厂过度累积；在 240 个测试实例上，GDRL 在有效性、稳定性和泛化能力方面超越了 8 种经典调度规则、3 种已

有 RL 方法和 5 种元启发式算法。Song 等人^[48] (2022) 在 *IEEE Transactions on Industrial Informatics* 上发表了基于 GNN 和 DRL 的柔性作业车间调度方法, 通过析取图表示调度状态并利用图神经网络提取结构化特征。值得注意的是, GNN 的引入虽然增强了对图结构的建模能力, 但并不能替代对非图类特征 (如资源负载、时间统计量等) 的工程化处理, 这从侧面印证了特征工程与深度学习模型结合使用的必要性。

(2) 经验回放机制改进。标准 DDQN 使用均匀采样的经验回放, 但不同经验样本对学习的重要性差异很大。Li 等人^[49] (2022) 提出了基于调度关键路径的优先经验回放机制, 优先学习对 workflow 完成时间影响较大的关键任务调度决策, 加速了 DDQN 的收敛并提高了最终调度策略质量。Fan 等人^[45] (2022) 在其 DRAS 系统中也设计了基于集群负载变化的分层经验回放策略, 使智能体能够更有效地学习不同负载条件下的调度模式。

(3) 探索策略改进。云调度环境的复杂性要求更有效的探索策略。Zhang 等人^[50] (2024) 设计了自适应 ϵ -贪心策略, 根据 workflow 执行阶段和系统负载动态调整探索率。文献^[51]通过 Boltzmann 动作探索策略增强了智能体的探索能力, 并在多数据中心任务调度场景下提出了成本感知的动作策略。Wu 等人^[52] (2024) 针对处理时间不确定的动态作业车间调度问题, 提出了混合优先经验回放 (hybrid prioritized experience replay) 机制, 优先重放具有正 TD 误差的经验样本, 使 PPO 智能体在不确定性环境中获得更具针对性的学习信号; 该方法在经典基准上比已有最优 DRL 方法平均降低 1.59% 的 Makespan, 且已训练策略在新不确定性条件下复用时可减少 27% 的训练时间, 为云调度中的不确定性处理提供了重要参考。

(4) 混合方法与算法融合。近年来, 将 DRL 与其他优化技术相融合成为一个重要趋势。Zhang 等人^[39] (2023) 将遗传算法的全局搜索能力与 DRL 的在线决策能力相结合, 用于实时 workflow 的成本感知调度。Lei 等人^[53] (2022) 在 *Expert Systems With Applications* 上提出了 Multi-PPO 框架, 创新性地设计了多指针图网络 (Multi-Pointer Graph Networks, MPGN) 架构——包含两个编码器-解码器组件, 分别学习作业操作选择子策略和机器分配子策略; 通过将柔性作业车间调度问题建模为多重马尔可夫决策过程 (MMDP), 允许智能体在每个决策步骤中同时做出多个调度动作, 在 2000 操作规模的实例上仍能保持良好的泛化性能, 运行效率显著优于元启发式算法。Yuan 等人^[54] (2024) 在 *Expert Systems With*

Applications 上提出了轻量级 MLP 状态嵌入方法替代复杂的 GNN，设计了直接反映候选动作特征的状态表示，用单一动作空间和单一策略模型同时解决 FJSSP 的两个子问题，在 Barnes、Brandimarte 和 Hurink 等经典基准上实现了优化能力与计算效率的更优平衡——这一结果有力地表明，精心设计的特征工程可以达到甚至超越复杂图神经网络的性能，与本研究 FE-IDDQN 的核心思路高度一致。

(5) 多智能体协作。对于大规模云环境，单一 DDQN 智能体可能难以有效处理全局调度问题。孙长银和穆朝絮^[54]、(2020) 探讨了多智能体深度强化学习的可能性，提出通过多个专门化的 DDQN 智能体协作完成复杂调度任务。Zhang 等人^[55] (2022) 在 IEEE Transactions on Industrial Informatics 上提出了基于 Dueling DQN (D3QN) 的分布式实时调度方法 (DRTS)，设计了云-边协作架构——在边缘侧部署分布式 Actor 进行实时决策，在云端部署集中式 Learner 进行模型训练，通过半马尔可夫决策过程 (semi-MDP) 同时处理加工排序和物流分配两类调度决策。实验表明，D3QN 在频繁作业到达场景下将加权迟到率降低了 5.29%，流程时间显著短于遗传算法和分层贪心策略，为大规模分布式云调度场景提供了有效的参考架构。

2.2.3 基于特征工程的 DRL 调度研究

本节从文献角度回顾特征工程在 DRL 调度中的具体应用进展。

(1) 多维度特征融合。孙恩昌^[56]等人 (2023) 在研究深度强化学习与移动通信资源管理时，强调了多维度特征融合的重要性，提出了从任务特性、资源状态和系统环境三个层面构建综合特征表示的层次化框架。实验表明，这种多维度特征融合能够使 DRL 智能体更全面地理解调度环境，做出更优的决策。Chen 等人^[18] (2023) 在其 HCDRL 异构 workflow 协同调度方法中，设计了结构特征 (任务依赖深度、并行度、关键路径位置) 和时序特征 (历史执行时间分布、资源使用模式) 的联合编码方案，构建了能够反映异构环境复杂性的综合状态表示；实验结果显示，任务加载策略下 Makespan 改善 16.6%、成本降低 16.3%、公平性指数提升 5.3%，验证了多维度特征融合对调度性能的显著贡献。Wang 等人^[27]

(2024) 在 DRLIS 边缘-雾计算调度方法中提出了分层特征表示方法，将边缘层和云层的资源状态分别编码后再进行融合，在 FogBus2 框架的实际部署中将加权成本降低了 50%，有效捕捉了边缘-云协同环境的层次化特性。

(2) 时序特征建模。云工作流的执行具有明显的时序特性。Wang 等人^[57] (2024) 提出将长短期记忆网络 (LSTM) 与 DDQN 结合, 捕捉工作流执行过程中的时序模式, 使调度智能体能够更好地预测任务执行时间和资源需求, 从而做出更准确的调度决策。Zhang 等人^[58] (2023) 在动态柔性作业车间调度中引入了可变处理时间 (VPT-FJSP) 模型, 利用 PPO 算法和图神经网络嵌入加工信息矩阵和车间状态, 并通过基于熵加权的动作空间预取策略加速收敛; 相比静态调度实现了 94.29% 的优化率, 在动态 VPT 环境下展现了强鲁棒性和泛化能力, 为时序不确定性建模提供了重要参考。Wang 和 Liao^[59] (2024) 在 *Journal of Intelligent Manufacturing* 上将离散事件仿真 (DES) 与 PPO 相结合, 构建了事件驱动的调度智能体训练框架, 在每个决策点提取基于事件序列的状态特征 (包括机器空闲率、加工队列长度变化等), 实现了在线实时调度决策, 为利用历史运行数据构建时序特征提供了可操作的方法论。

(3) 图结构特征表示。工作流本质上是一个 DAG 结构, 从图结构中提取特征成为一个活跃的研究方向。Chen 等人^[46] (2023) 探索了使用图嵌入技术将工作流的拓扑结构转化为低维向量表示, 作为 DRL 智能体的输入特征, 显著提升了调度性能, 特别是对于结构复杂的工作流。Huang 等人^[47] (2024) 发展了基于 GNN 的端到端特征学习方法, 利用消息传递机制自动聚合邻居节点信息, 构建能够编码全局拓扑结构的任务嵌入向量。Song 等人^[48] (2022) 基于析取图表示方法, 设计了能够同时编码工艺约束和时序信息的图特征, 为复杂调度问题的状态表示提供了新范式。Koslovski 等人^[41] (2024) 在 DAG 工作流调度中将 DAG 的拓扑属性 (如关键路径长度、层次结构、并行度等) 编码为状态向量的一部分, 有效提升了 Actor-Critic 智能体对工作流结构信息的理解能力。

(4) 动态与不确定性感知特征。在实际云环境中, 任务的执行时间和资源需求往往具有不确定性。Wu 等人^[52] (2024) 在 *Engineering Applications of Artificial Intelligence* 上提出了面向不确定处理时间的 PPO 调度模型, 创新性地设计了基于可行解矩阵 (feasible solution matrix) 的状态表示来描述调度顺序, 并提出了混合优先经验回放机制 (优先重放具有正 TD 误差的样本); 实验表明, 该方法在经典基准上平均 Makespan 降低 1.59%, 且已训练策略在不确定性环境下复用时可减少 27% 的训练时间。Chang 等人^[60] (2022) 设计了包含作业到达率估计和队列状态变化趋势的动态特征, 使智能体能够适应非稳态的任务到达模式。

Gui 等人^[61] (2023) 引入了基于滑动窗口的动态特征提取方法, 捕捉系统状态的短期变化趋势。

值得指出的是, DRL 在制造业调度、能源系统调度和通信网络调度等相关领域也得到了广泛应用, 为云 workflow 调度提供了重要的方法论借鉴。在制造业调度方面, Lee 等人^[62] (2022) 在 *Expert Systems With Applications* 上将 DQN 应用于半导体制造调度, 创新性地设计了包含生产计划偏差特征的状态表示和集中式神经网络架构以适应可变机器数量, DRL 在多种调度场景下优于 FIFO、EDD、SPT 等规则; Du 和 Li^[63] (2024) 在 *International Journal of Production Economics* 上提出了 DQN-EA-SR 方法, 通过三个协调的 Double DQN (选择 DQN、局部 DQN、全局 DQN) 构建层次化调度决策拓扑, 处理预制混凝土生产中的分布式 FJSP 问题。在能源系统调度方面, Zhao 等人^[64] (2022) 在 *Applied Energy* 上提出了基于 PPO 的家庭多能源系统联合负荷调度方法, 通过混合分布同时处理连续动作 (功率可调设备) 和离散动作 (时间可调设备), 平均用能成本比 DQN 降低 12.17%; Park 和 Moon^[65] (2022) 在 *Applied Energy* 上首次将多智能体 DRL (MADRL) 应用于智能电网中多电动汽车充放电调度, 采用 COMA 策略梯度和 MADDPG 的集中训练分散执行 (CTDE) 架构; Li 等人^[66] (2025) 在 *Applied Energy* 上提出了 SA-SAC (State-Adversarial Soft Actor-Critic) 算法, 通过对抗训练增强综合能源系统调度在网络攻击下的鲁棒性, 在攻击场景下经济性能提升 10%。在通信网络调度方面, Yu 等人^[67] (2023) 在 *IEEE Transactions on Industrial Informatics* 上提出了 Deep-DFS 确定性流调度器, 通过延迟感知网络表示、动作掩码机制和关键性感知奖励函数, 实现了混合关键性流量的高效调度。这些领域在混合动作空间处理^[55]、多智能体协作^[55] ^[55]、对抗鲁棒性^[66]、层次化 DQN 架构^[55]和动作掩码^[55]等方面积累的经验, 以及其他相关研究^[55, 68] 在特征工程方法和训练策略方面的探索, 均可迁移应用于云 workflow 调度系统的设计与优化。

综合上述文献分析可以发现, 当前 DRL 调度研究中的特征工程实践多为面向特定任务的局部设计, 缺乏系统性的多层次特征提取与融合方法论。特别是在企业级调度平台上, 如何从海量历史运行数据中系统性地挖掘和构建对调度决策最为关键的特征, 仍是一个亟待深入研究的问题。

2.3 现有研究的不足与本文定位

尽管 DRL 在云 workflow 调度领域取得了显著进展，但现有研究仍存在以下重要的局限性和挑战：

1. 状态表示不足。标准 DDQN 通常使用相对简单的状态表示，难以充分捕捉 workflow 调度环境中的复杂特征和模式，如任务依赖关系、资源异构性、历史执行模式等。Zhou 等人^[22]在综述中指出，如何构建既能充分表达调度环境信息又不至于高维的状态表示，仍然是一个开放性問題。

2. 样本效率低。在复杂的调度环境中，经验样本的获取成本高，而标准 DDQN 的样本利用效率不高。Fan 等人^[45]在其 DRAS 系统研究中指出，每个训练样本的获取需要完整的作业执行过程，训练效率成为制约 DRL 调度方法实用化的重要瓶颈。

3. 探索-利用平衡不佳。标准的 ϵ -greedy 探索策略在大型动作空间中效率低下。尽管 Wu 等人^[52]分别提出了自适应探索和不确定性感知探索策略，但在更大规模的调度问题中，有效的探索策略设计仍面临巨大挑战。

4. 网络结构不适应。通用的全连接网络结构难以有效处理 workflow 调度中的结构化信息。虽然 GNN^[47]^[33]的引入在一定程度上缓解了这一问题，但如何设计能够同时处理图结构、时序信息和数值特征的统一网络架构，仍需进一步研究。

5. 缺乏领域知识整合。标准 DDQN 是一个通用框架，没有整合调度领域的专业知识，导致学习效率低下或陷入次优解。Yuan 等人^[54]和 Lei 等人^[53]分别尝试将调度规则知识融入 DRL 框架，但系统性地整合调度领域知识的方法论仍不成熟。

6. 实际应用验证不足。Gu 等人^[33]（2025）在其综述中指出，大多数 DRL 调度研究仍停留在仿真环境或小规模测试环境中，很少有研究在真实的、大规模的生产环境中进行验证。Hou 等人^[23]也强调了从模拟环境到真实云平台的部署挑战。

7. 特征工程系统性不足。虽然特征工程的重要性得到了越来越多的认可，但如 2.3.3 节所述，现有研究中对特征的提取多为面向特定场景的局部设计，缺乏从任务级、工作流级到资源环境级的系统性多层次特征工程方法论。特别是如何从企业级调度平台的海量历史运行数据中挖掘对调度决策最关键的特征，仍是一个亟待解决的问题。

8. 泛化能力有限。Mangalampalli 等人^[35]的研究指出，现有的 DRL 调度算法面对新的 workflow 类型或资源配置时泛化能力往往有限。

针对上述不足，本研究的定位和主要工作体现在以下几个方面：第一，提出深度特征工程驱动的改进 DDQN 调度方法（FE-DDQN），通过系统性地从 DolphinScheduler 的海量历史运行数据中提取多层次、多维度的调度特征，构建信息丰富的状态表示，以弥补现有研究在状态表示方面的不足（对应上述局限性 1 和 7）；第二，对 DDQN 算法进行针对性改进，包括网络结构优化和训练策略改进，以更好地适应云 workflow 调度场景的特殊需求（对应局限性 4 和 5）；第三，在真实的企业级调度平台——中国人寿 DolphinScheduler 生产环境上进行系统性验证，弥合理论研究与实际应用之间的鸿沟（对应局限性 6）。

2.4 本章小结

本章从三个方面对本研究的相关工作进行了系统性梳理。首先，概述了云 workflow 调度的问题特性、关键评价指标和本研究的实践平台 Apache DolphinScheduler（2.1 节）。其次，全面回顾了 DRL 在云 workflow 调度中的研究现状（2.3 节），涵盖 DRL 调度方法、DDQN 算法改进和基于特征工程的 DRL 调度研究三个维度的最新进展，最后，分析了现有研究的不足，明确了本研究通过 FE-DDQN 方法弥补上述不足的定位和思路。上述梳理为后续章节的环境建模、特征工程设计、算法改进和实验验证提供了理论依据和方法论指导。

第 3 章 面向云 workflow 调度的特征工程与强化学习环境设计

本章详细阐述如何从原始的云 workflow 运行数据中提取有效特征，并基于这些特征构建一个能够支持改进 DDQN 算法训练的强化学习环境。首先，本研究将对云 workflow 调度问题进行形式化定义，并构建包含执行时间和资源利用率的多目标优化函数。接着，重点介绍运行数据的特征工程过程，包括数据来源分析、预处理方法、多维度特征（任务级、workflow 级、资源环境级）的提取、选择与表示策略。最后，基于构建的特征集，详细设计强化学习智能体的状态空间、动作空间以及能够准确反映调度优化目标的奖励函数。

3.1 云 workflow 调度问题的定义

3.1.1 工作流与资源模型

一个工作流实例可以表示为一个有向无环图， $W = (T, E)$ ，其中 $T = \{t_1, t_2, \dots, t_n\}$ ，是工作流中所有任务的集合， n 是任务的总数量。每个任务 t_n 具有特定的计算需求（如预期的 CPU 周期、内存需求）和可能的输入输出数据。 E 是任务之间依赖关系的集合。一个依赖关系 $(t_i, t_j) \in E$ 表示任务 t_i 是任务 t_j 的直接前驱， t_j 必须在 t_i 完成后才能开始执行。没有前驱任务的称为入口任务，没有后继任务的称为出口任务。对于每个任务 t_i ，本研究可以定义其属性， P_i 任务 t_i 的执行时间， mem_i 表示任务 t_i 执行所需的内存大小。 $input_data_i$ 任务 t_i 的输入数据集大小。 out_data_i 任务 t_i 的输出数据集大小。

云计算环境提供了一系列异构的计算资源，可以表示为 $R = \{r_1, r_2, \dots, r_m\}$ ，其中 m 是可用计算资源（如虚拟机、容器实例或物理 Worker 节点）的总数量。每个资源 r_j 具有其自身的特性， $speed_j$ ：资源 r_j 的处理速度， $cores_j$ ：资源 r_j 的 CPU 核心数。 $avali_mem_j$ ：资源 r_j 的可用内存大小。资源的状态是动态变化的。

调度过程的核心是将工作流中的每个任务 t_i 分配给一个合适的资源 r_j ，并确定其开始执行时间 st_{ti} 和预计完成时间 ft_{ti} 。调度决策需要满足任务间的依赖约束，即对于任意 $(t_i, t_j) \in E$ ，必须有 $ft_{ti} \leq st_{tj}$ 。同时，在任何时刻，分配给同一资源的任务所消耗的总资源量不能超过该资源的容量。

3.1.2 多目标优化函数构建

本研究的核心优化目标是最小化工作流的总完成时间（Makespan）并最大化计算资源的平均利用率。最小化Makespan ($T_{makespan}$), Makespan 定义为工作流中所有任务完成时间的最大值减去最早开始任务的开始时间。更准确地说, 是最后一个出口任务的完成时间。

$$T_{makespan} = \max_{t_j \in T_{exit}} \{ft_{ti}\} \quad (3.1)$$

其中 T_{exit} 是所有出口任务的集合。本研究的目标是 $\min (T_{makespan})$ 。

2. 最大化平均资源利用率 U_{avg} , 资源利用率针对 CPU 和内存分别考虑, 然后综合评估平均 CPU 利用率

$$U_{cpu_avg} = \frac{\sum_{j=1}^m \sum_{t=0}^{T_{makespan}} cpu_used_j(t)}{\sum_{j=1}^m cores_j \cdot T_{makespan}} \quad (3.2)$$

其中 $\sum_{j=1}^m \sum_{t=0}^{T_{makespan}} cpu_used_j(t)$ 是表示所有资源在所有时刻被占用的 CPU 核心数之和。

平均内存利用率

$$U_{mem_avg} = \frac{\sum_{j=1}^m \sum_{t=0}^{T_{makespan}} mem_used_j(t)}{\sum_{j=1}^m total_mem_j \cdot T_{makespan}} \quad (3.3)$$

其中 $mem_used_j(t)$ 是资源 r_j 在时刻 t 被占用的内存量, $total_mem_j$ 是资源 r_j 的总内存。本研究的目标是 $\max (U_{cpu_avg})$ 和 $\max (U_{mem_avg})$ 。在强化学习的框架下, 这两个目标需要被整合到奖励函数的设计中, 奖励函数设计为资源利用率等要素的加权和。

$$w1 * (-T_{makespan}) + w2 * U_{cpu_avg} + w3 * U_{mem_avg} \quad (3.4)$$

其中 $w1, w2, w3$ 是权重因子, 表示不同目标的重要性。权重的选择对最终的调度策略有显著影响, 需要根据实际需求进行调整和实验。

此外, 调度还需要满足约束条件。在任何时刻 t , 分配给资源 r_j 的所有并发任务的总 CPU 需求不能超过 $cores_j$, 总内存需求不能超过其可用内存。

$$ft_{ti} \leq st_{tj}, (t_i, t_j) \in E \quad (3.5)$$

3.2 基于 DolphinScheduler 运行数据的特征工程

有效的特征工程是构建高性能深度强化学习调度器的基石。本节将详细阐述从 workflow 及任务实例数据中提取、构建和选择对调度决策至关重要的特征。

本研究将遵循系统性的特征工程流程，确保所选特征能够全面、准确地刻画任务、workflow 及资源环境的动态特性。

3.2.1 数据来源分析与预处理

数据主要来源于某金融企业生产环境的 DolphinScheduler 的后端 oceanbase 数据库，包括监管报送、商务智能、客户服务、财务处理、契约、核保业务等工作流执行数据，从 2024 年 6 月到 2024 年 12 月，共储备了 510 万条工作流实例运行数据以及 9100 万条任务实例数据，数据为每天定时存储累计而来，覆盖业务高峰期和平峰期数据。该数据集完整记录了动态异构云环境下任务调度的全生命周期信息，包括资源分配、依赖关系、执行状态、任务类型等。由于数据量非常大，包含的业务种类多，为了平衡计算效率与数据完整性，采用分层抽样的办法进行抽样，同时兼顾任务类型的多样性。

数据库中的核心表通过多表关联构建端到端的知识图谱，关键表结构如下表，数据通过主键-外键进行关联，实现业务的全流程追踪，从上到下可以理解为项目-workflow-任务这三个级别。

表 3.1 workflow 核心表结构

表名	核心字段功能
t_ds_process_instance	state（状态码）、start_time/end_time（执行耗时）host（运行节点）、command_param（动态参数）
t_ds_task_instance	task_type（任务类型）、host（资源分配）、retry_times（重试次数）、task_params（运行时参数）
t_ds_process_task_relation	pre_task_code/post_task_code（DAG 依赖关系）、condition_type（分支条件逻辑）
t_ds_task_definition	timeout（超时阈值）、resource_ids（绑定资源）、task_priority（优先级配置）

由于数据中存在一些异常数据，比如由于用户主动停止的工作流、或者由于网络原因、服务器重启等导致运行失败的工作流，或者存在运行时间过长的工作流，比如运行超过一天的工作流实例，所以对数据进行清洗，清洗之后只留下成功状态的工作流实例和任务实例。

表 3.2 workflow 状态和任务状态

workflow 或者任务状态字段	状态解释
0AMD12321312AA	提交成功

1	正常运行
2	准备暂停
3	暂停
4	准备停止
5	停止
6	失败
7	成功
8	需要容错
9	杀掉进程
10	等待线程
11	等待依赖完成

表 3.3 任务和工作流优先级

工作流或者任务优先级	中文含义
0	最高
1	高
2	中
3	低
4	最低

数据中包含多种的任务类型，因为在实际的生产环境中，用户需要处理不同类型的数据，同时也可能对数据进行不同类型的处理，有可能使用自己开发的 jar 包，任务类型及特性如下表

表 3.4 任务节点类型

任务节点类型	特点
SQL	耗时：短查询（秒级）到长查询（小时级），取决于 SQL 复杂度与数据量。资源需求：轻度至重度（与查询涉及的计算逻辑相关，如多表 JOIN、窗口函数）。场景：数据查询、表结构变更、ETL 中间步骤等。注意事项：索引优化和 SQL 调优可大幅提升效率
HTTP	耗时：取决于接口响应速度（受网络延迟和服务器处理能力影响）。资源需求：低（仅在调用阶段占用少量网络资源）。场景：触发 API、通知告警、触发下游服务等。注意事项：需处理超时、重试及异常状态码。

SQOOP	耗时：数据量越大耗时越长（IO 密集型），尤其全量同步时。资源依赖：依赖 Hadoop 集群资源（MapReduce/YARN）及数据库连接池性能。场景：关系型数据库（MySQL/Oracle）与 Hadoop（HDFS/Hive/HBase）间的数据迁移。注意事项：网络带宽和数据库并发能力是关键限制因素
DATAX	耗时：数据量大的同步场景耗时较长（类似 SQOOP）。资源需求：高（需要足够内存和 IO 带宽）。场景：异构数据源（如 MySQL→HDFS、Hive→Oracle）同步。注意事项：配置合理的并发度和切分键可提升效率。
DEPENDENT	耗时：无实际执行耗时，仅检查依赖条件。资源需求：无实际资源消耗。
PROCEDURE	耗时：与存储过程逻辑复杂度相关（可能涉及多步骤数据库操作）。资源需求：依赖数据库资源（如 Oracle PL/SQL 的 PGA 内存分配）。场景：数据库内复杂业务逻辑（如批量数据处理、事务管理）
SHELL	耗时：短至中（执行脚本文件或命令）。资源需求：低（简单脚本）至高（脚本调用外部计算工具）。

除了 DolphinScheduler 自身的元数据，还整合了来自外部监控系统 Prometheus 的 CPU、内存、网络 I/O、磁盘 I/O 资源利用率数据，以及 Kubernetes 资源管理平台的资源分配信息。

原始数据在直接用于特征提取前，需要进行预处理。剔除关键时间戳丢失、状态异常的、任务状态与开始/结束时间逻辑不符记录。将来自不同表（如 `t_ds_process_instance`, `t_ds_task_instance`, `t_ds_process_definition`）的数据通过关联键连接起来，形成包含工作流和任务完整信息的宽表，便于后续特征提取。计算任务执行时长 (`end_time - start_time`)，任务等待时长 (`start_time - submit_time`)。解析 DAG 结构、`task_params` 等字段获取任务依赖关系、任务类型等信息。在模型初步训练和验证阶段，需要根据特定场景（如用户提到的监管报送、反洗钱场景的工作流）或时间范围进行采样，以减少计算开销并快速迭代。但最终模型应尽可能利用全量数据进行训练，以保证泛化能力。

3.2.2 多维度特征提取与构建

基于预处理后的数据，本研究可以从任务级、workflow 级和资源环境级三个主要维度提取和构建特征。这些特征将共同构成强化学习智能体的状态表示。任务级特征描述了待调度任务或正在运行任务的个体属性。

表 3.5 任务级特征

task_type	任务类型, SHELL, PYTHON, SPARK, SQL 等
task_priority	任务优先级
declared_cpu_cores	任务声明的 CPU 核心数, 估算数据
declared_memory_mb	任务消耗内存, 估算数据
is_io_intensive	Io 敏感
is_cpu_intensive	Cpu 敏感
is_memory_intensive	内存敏感
historical_avg_duration_seconds	历史平均执行时长 (秒), 近 N 次均值
historical_avg_cpu_usage_percent	任务定义的历史平均 CPU 使用率
historical_avg_memory_usage_mb	任务定义的历史平均内存使用峰值
historical_success_rate	该任务定义的历史执行成功率
estimated_remaining_time_seconds	估计剩余执行时间, 历史平均时长和已执行时长计算
current_wait_time_seconds	对于待调度任务, 当前已等待时间
num_direct_predecessors	直接前驱任务数量
num_direct_successors	直接后继任务数量
num_total_predecessors	所有前驱任务数量 (间接+直接)
num_total_successors	所有后继任务数量
dag_depth_level	任务在 DAG 中的深度
is_on_critical_path	任务是否位于当前 workflow 实例的关键路径上 (需要动态计算)
subgraph_complexity_ahead	任务后续子图的复杂度

表 3.6 workflow 级特征 (Workflow-level Features)

workflow_total_tasks	workflow 定义的总任务数
workflow_max_parallelism	workflow 定义的最大并行度 (DAG 宽度)
workflow_critical_path_length_static	workflow 定义的静态关键路径长度
workflow_type_encoded:	workflow 类型或所属业务域, 监管报送、反洗钱

第 3 章 面向云 workflow 调度的特征工程与强化学习环境设计

workflow_progress_percent	当前 workflow 实例已完成任务的百分比
workflow_remaining_tasks	当前 workflow 实例剩余待执行和正在执行的任务数
workflow_current_makespan_seconds	当前 workflow 实例已耗时
workflow_tasks_in_ready_queue	当前处于就绪状态（依赖满足，等待资源）的任务数量
workflow_tasks_running	当前正在运行的任务数量
workflow_tasks_failed	当前已失败的任务数量
workflow_avg_task_wait_time_seconds	当前 workflow 实例中已完成任务的平均等待时间

表 3.7 资源环境特征

worker_current_cpu_util_percent_rt	Worker 节点当前的实时 CPU 利用率
worker_avg_cpu_util_percent_hist_short	Worker 节点短期（如过去 5 分钟）平均 CPU 利用率
worker_avg_cpu_util_percent_hist_long	Worker 节点长期（如过去 1 小时）平均 CPU 利用率
worker_current_mem_util_percent_rt	Worker 节点当前的实时内存利用率
worker_avg_mem_util_percent_hist_short	Worker 节点短期平均内存利用率
worker_avg_mem_util_percent_hist_long	Worker 节点长期平均内存利用率
worker_available_cpu_cores_rt	Worker 节点当前可用 CPU 核心数
worker_available_memory_mb_rt	Worker 节点当前可用内存
worker_num_running_tasks_rt	Worker 节点当前正在运行的任务数量
worker_disk_io_util_percent_rt	Worker 节点当前磁盘 I/O 利用率
worker_network_io_bps_rt	Worker 节点当前网络吞吐量
worker_type_encoded	Worker 节点类型或配置
worker_group_id	所属 Worker 组 ID
cluster_total_available_cpu_cores	集群总可用 CPU 核心数
cluster_total_available_memory_mb	集群总可用内存
cluster_avg_cpu_util_percent	集群平均 CPU 利用率
cluster_avg_mem_util_percent	集群平均内存利用率
num_idle_workers	空闲 Worker 节点数量
num_busy_workers	高负载 Worker 节点数量
global_task_queue_length	全局任务等待队列长度

历史平均类特征，选择一天时间窗口进行统计，不同特征的取值范围可能差异很大，需要 Min-Max Scaling 到[0,1]进行归一化，以利于神经网络的训练。使用独热编码或嵌入层（Embedding Layer）处理任务类型、Worker 类型等类别特征。许多特征（如资源利用率、任务等待时间）是动态变化的，需要在每个调度决策点实时或准实时地更新。

3.2.3 特征选择与表示

特征选择方法包括过滤法 (Filter Methods)、包裹法 (Wrapper Methods)和嵌入法 (Embedded Methods)。考虑到 DDQN 模型的复杂性，先通过领域知识和初步的过滤法筛选掉明显不相关的特征，然后依赖深度神经网络自身的学习能力来捕捉重要特征的组合和非线性关系。在模型训练后，通过分析网络权重或使用 SHAP (SHapley Additive exPlanations) 等可解释性工具来评估特征的重要性，并进行迭代优化。

最终选定的特征集将组合成一个状态向量 (State Vector) St ，作为 DDQN 网络的输入。本研究中，包含多个待调度任务和多个可用 Worker，将每个待调度任务的特征向量化 V_{task_i} ，将每个可用 Worker 的特征向量化 V_{worker_j} ，将全局 workflow 和集群特征向量化： V_{global} 。然后将这些向量拼接或通过更复杂的结构（如图神经网络、注意力机制）组合成最终的状态表示。如果一次只为一个任务做决策（选择哪个 Worker），那么状态可以主要围绕该任务的特征和所有可用 Worker 的特征展开。特征工程是一个迭代的过程，需要结合领域知识、数据分析和模型实验不断优化，以找到最能提升调度性能的特征组合与表示方式。

3.3 强化学习环境设计：状态、动作与奖励

基于这些特征设计一个合适的强化学习（RL）环境。该环境将作为智能体（Agent，即本研究的 DDQN 调度器）学习和交互的平台。一个 RL 环境主要由状态空间（State Space）、动作空间（Action Space）和奖励函数（Reward Function）定义。

3.3.1 状态空间（State Space）的定义

状态空间 S 定义了智能体在做出决策时能够观测到的所有环境信息。一个精心设计的状态表示应该简洁、信息丰富，并且满足马尔可夫性质，即当前状态足以做出最优决策，无需依赖更早的历史信息。基于 3.2 节提取的多维度特征，本研究可以构建状态向量。

在每个调度决策点，智能体需要为某个待调度任务选择一个执行资源。因此，状态表示需要包含当前待决策任务的信息、所有可用资源的信息以及全局系统/workflow 的信息。

状态空间 S 由三部分组成：待调度任务特征、可用 Worker 特征和全局环境特征。形式化定义如下：

$$S = \{V_{task_to_schedule}, V_{available_workers}, V_{global}\}$$

$$V_{task_to_schedule} = [v_{task_1}, v_{task_2}, \dots, v_{task_n}]$$

$$v_{task_i} =$$

$$[CPU_req_i, Mem_req_i, Est_time_i, Priority_i, In_degree_i, Out_degree_i, Critical_path_pos_i, current_wait_time]$$

$$V_{available_workers} = [v_{worker_1}, v_{worker_2}, \dots, v_{worker_m}]$$

$$v_{worker_j} =$$

$$[CPU_{util_j}, Mem_{util_j}, Avail_{CPU_j}, Avail_{Mem_j}, Queue_{len_j}, Load_{trend_j}, running_{task_{num}}, worker_type]$$

$$V_{global_env} =$$

$$[Workflow_progress, Cluster_CPU_util, Cluster_Mem_util, Resource_balance, Peak_indicator]$$

$Peak_indicator$ 是否高峰期标志。

假设系统中有 n 个待调度任务，每个任务有 d_t 维特征； m 个可用 Worker，每个 Worker 有 d_w 维特征；全局环境有 d_g 维特征。则状态空间的总维度为：

$$dim(S) = n \times d_t + m \times d_w + d_g$$

3.3.2 动作空间（Action Space）的定义

动作空间 A 定义了智能体在每个状态下可以采取的所有可能行动。在 workflow 调度问题中，一个核心动作是将一个待调度任务分配给一个特定的可用计算资源。

对于本研究，本研究定义一个离散的动作空间。如果系统中有 m 个可用的 Worker 节点（或资源槽位），那么当有一个任务 t_i 需要调度时，智能体可以选择将 t_i 分配给这 m 个 Worker 中的任意一个。

$A = \{a_1, a_2, \dots, a_j\}$ 。其中， a_j 表示将当前待调度任务分配给第 j 个 Worker 节点。

如果某个 Worker j 的可用内存小于任务 t_i 所需的内存，则将 t_i 分配给 Worker j 是一个非法动作。如果某个 Worker j 的可用 CPU 核心数不足，也构成

非法动作。在 DDQN 算法中，对于非法动作，可以将其 Q 值设置为一个非常小的负数（如 $-\infty$ ），从而确保智能体不会选择它们。或者，在动作选择阶段，只从合法的动作子集中进行选择。

在某些复杂的调度场景中，可能还需要一个特殊的“不立即调度”或“延迟调度”的动作，特别是当所有可用资源都处于高负载状态，或者等待更好的调度时机可能更有利时。但这会增加动作空间的复杂性。在本研究初期，本研究可以先不考虑这个动作，假设一旦任务就绪且有资源满足其基本需求，就必须进行调度。

在实际的生产作业中，是对 worker 进行分组处理的。动作空间的大小等于可用 Worker 的数量 m 。如果 m 非常大，可能会导致 Q 值学习困难。这样分组处理后，会缓解这种现象。

3.3.3 奖励函数的设计

本研究提出的多目标奖励函数 R_t 在时间步 t 定义如下：

$$R_t = w_1 \cdot R_{\text{makespan}} + w_2 \cdot R_{\text{resource util}} + w_3 \cdot R_{\text{balance}}$$

其中 w_1, w_2, w_3 是权重系数，满足 $w_1 + w_2 + w_3 = 1$ ，用于平衡不同优化目标的重要性。通过大量实验和参数调优，确定最优权重组合为 $w_1=0.6, w_2=0.3, w_3=0.1$ 。

执行时间奖励定义为：

$$R_{\text{makespan}} = \begin{cases} \alpha \cdot \frac{ET_{\text{est}} - ET_{\text{actual}}}{ET_{\text{est}}} + \beta \cdot CP_{\text{impact}}, & \text{if task completed} \\ \gamma \cdot \frac{WT_{\text{avg}} - WT_{\text{current}}}{WT_{\text{avg}}}, & \text{if task waiting} \\ -\delta, & \text{if task failed} \end{cases}$$

其中 ET_{est} 是任务的估计执行时间， ET_{actual} 是实际执行时间。 CP_{impact} 是任务对关键路径的影响因子，定义为：

$$CP_{\text{impact}} = \begin{cases} 1.0, & \text{if task on critical path} \\ 0.5, & \text{if task may affect critical path} \\ 0.2, & \text{otherwise} \end{cases}$$

WT_{avg} 是历史平均等待时间, $WT_{current}$ 是当前任务的等待时间。调节系数 $\alpha = 0.7, \beta = 0.3, \gamma = 0.5, \delta = 1.0$ 。

资源利用率奖励定义为

$$R_{resource\ util} = \lambda \cdot R_{cpuutil} + (1 - \lambda) \cdot R_{memutil}$$

其中 $\lambda=0.5$ 。CPU 和内存利用率奖励分别定义为

$$R_{cpuutil} = \begin{cases} \frac{U_{cpu} - U_{cpu_{min}}}{U_{cpu_{opt}} - U_{cpu_{min}}}, & \text{if } U_{cpu_{min}} \leq U_{cpu} \leq U_{cpu_{opt}} \\ \frac{U_{cpu_{max}} - U_{cpu}}{U_{cpu_{max}} - U_{cpu_{opt}}}, & \text{if } U_{cpu_{opt}} < U_{cpu} \leq U_{cpu_{max}} \\ -\mu \cdot \frac{U_{cpu} - U_{cpu_{max}}}{1 - U_{cpu_{max}}}, & \text{if } U_{cpu} > U_{cpu_{max}} \\ -\nu \cdot \frac{U_{cpu_{min}} - U_{cpu}}{U_{cpu_{min}}}, & \text{if } U_{cpu} < U_{cpu_{min}} \end{cases}$$

内存利用率奖励 R_{mem_util} 采用类似的分段函数。其中:

- U_{cpu} 和 U_{mem} 是当前 CPU 和内存利用率。
- $U_{cpu_min}=U_{mem_min}=0.3$ (最小目标利用率)。
- $U_{cpu_opt}=U_{mem_opt}=0.75$ (最优目标利用率)。
- $U_{cpu_max}=U_{mem_max}=0.9$ (最大目标利用率)。
- 惩罚系数 $\mu=1.5, \nu=1.0$

负载均衡奖励定义为

$$R_{balance} = -\sigma \cdot \sqrt{\frac{1}{m} \sum_{j=1}^m (U_j - \bar{U})^2}$$

其中:

$\sigma=1.0$ (缩放因子)。

m 是 Worker 节点数量。

$$U_j = \frac{U_{cpu,j} + U_{mem,j}}{2} \text{ (第 } j \text{ 个 Worker 的综合资源利用率)}$$

$\bar{U}$ 是所有 Worker 的平均综合资源利用率。

奖励函数动态调整机制的形式化定义如下:

$$w_1(t) = w_1^{\text{base}} + \Delta w_1(WF_{\text{progress}}, System_{\text{load}}, Priority)$$

其中 WF_{progress} 是 workflow 完成进度, $System_{\text{load}}$ 是系统负载状态, $Priority$ 是 workflow 优先级。

在 workflow 调度中, 只有当整个 workflow 完全执行完毕后, 才能根据最终的执行时间、资源利用率等指标给出奖励, 这种稀疏的奖励机制使得智能体很难从中间步骤中学习到有效的策略, 因为它无法及时知道哪些动作是好的, 哪些是坏的。在 workflow 调度中, 一个任务的调度决策可能需要经过多个步骤后才能看到其对最终结果的影响, 这种延迟使得智能体难以将当前的动作与未来的奖励联系起来, 增加了学习的难度。在强化学习中, 智能体的学习过程依赖于奖励信号的反馈。如果奖励信号稀疏且延迟, 智能体可能会学习效率低下、难以收敛, 所以引入阶段性奖励和奖励塑形。塑形函数是

$$R'(s, a, s') = R(s, a, s') + \gamma \cdot \Phi(s') - \Phi(s)$$

$\Phi(s)$ 是一个潜在函数, $\Phi(s)$ 定义为 workflow 预期剩余执行时间的负值。这样, 智能体在每一步都会尝试减少预期剩余时间, 从而加速 workflow 的完成。阶段性奖励和奖励塑形都是为了缓解奖励稀疏性问题, 但它们的实现方式和理论基础不同。阶段性奖励依赖于任务的中间节点, 而奖励塑形通过势能函数提供额外的奖励信号, 使奖励更稠密且理论上不改变最优策略。

第4章 基于特征增强的改进 DDQN 调度算法

本章详细阐述基于特征增强的改进双重深度Q网络（Feature-Enhanced Improved Double Deep Q-Network，简称FE-IDDQN）调度算法。首先分析DDQN算法在云 workflow 调度中的应用潜力与挑战，然后系统介绍所提出的算法改进框架，包括特征增强机制、网络结构优化、经验回放机制改进以及探索策略调整等关键创新点。最后，给出算法的训练流程、参数设置和复杂度分析，并通过伪代码形式清晰呈现算法的实现细节。

4.1 FE-IDDQN 算法总体框架

FE-IDDQN算法是一个端到端的强化学习框架，专为解决异构云环境下的 workflow 调度优化问题而设计。其总体架构如图4-1所示，涵盖了特征增强机制、网络参数复制及改进的经验回放策略等核心模块。本文设计了三个并行的特征处理模块，分别处理任务特征、工作流特征和资源特征，每个模块使用独立的全连接层进行特征转换和提取，通过特征融合层将三类特征整合为统一表示。同时引入自注意力机制处理工作节点间的关系，使模型能够自动学习不同工作节点的重要性权重，显著提升了异构环境下的调度效果。

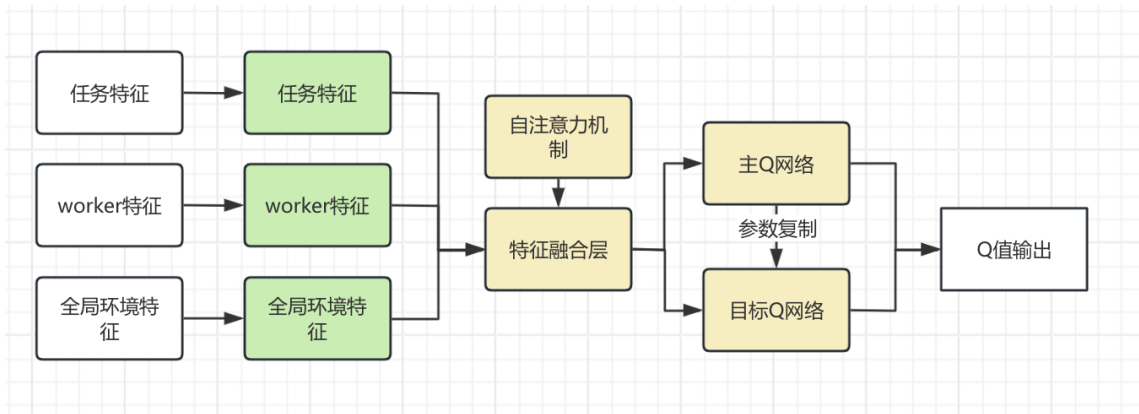


图 4.1 FE-IDDQN 算法框架图

4.1.1 算法核心组件

FE-IDDQN算法由特征工程模块、智能体模块、优先级经验回放缓冲区、自适应探索策略和奖励计算模块构成。

特征工程模块负责从原始系统数据中提取、转换和融合多维度特征，构建信息丰富的状态表示。FE-IDDQN智能体包含主网络和目标网络，负责学习状态到

动作的映射关系，即调度策略。优先级经验回放缓冲区存储和采样训练经验，优先考虑对学习更有价值的样本。自适应探索策略根据环境状态和学习进度动态调整探索行为。奖励计算模块是基于多目标优化函数计算调度决策的即时奖励。

4.1.2 算法工作流程

FE-DDQN 算法的运行机制由两个主要阶段构成：训练阶段与推理阶段。在训练阶段，智能体通过与环境的持续交互，不断优化其调度策略；而在推理阶段，智能体利用已收敛的网络参数，在实际应用场景中直接生成调度决策。

在训练过程中，首先从环境中获取当前状态信息，其中包括待调度任务与可用资源等要素。随后，特征工程模块对原始数据进行处理与抽取，得到结构化的状态表示。智能体在此基础上，依据自适应探索策略生成调度动作，将任务分配至具体资源节点。环境执行该动作并更新系统状态，同时返回奖励信号。所得经验元组被存入优先级经验回放缓冲区。智能体再从缓冲区中采样批量经验，用于更新 DDQN 网络参数，并在训练过程中定期同步目标网络及调整探索参数。以上过程不断迭代，直至模型收敛或达到预设训练轮次。

在推理阶段，系统首先采集当前状态，并通过特征工程模块转化为标准化的状态表示。随后，DDQN 网络直接依据贪婪策略输出调度决策。该决策被执行后，系统状态随之更新，从而实现工作流的动态调度。

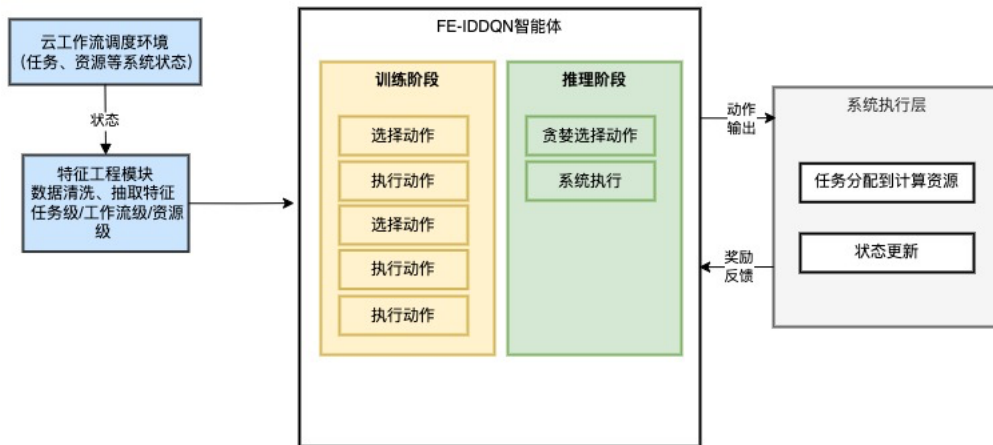


图 4.2 FE-DDQN 算法流程图

4.2 特征增强机制设计

特征增强是FE-DDQN算法的核心创新点之一，通过系统化的特征工程方法，从多个维度提取和融合关键信息，为DDQN智能体提供更丰富、更有区分度的状态表示。

4.2.1 多维度特征提取

FE-DDQN的特征提取框架包含四个关键维度：

任务特征向量 F_T ：

静态特征：任务类型、预估执行时间、资源需求（CPU、内存）、优先级等。

动态特征：任务等待时间、前驱任务完成比例等。

依赖特征：前驱任务数量、后继任务数量、在DAG中的层级等。

形式化表示：

$$F_T = [f_{t1}, f_{t2}, \dots, f_{tn}] \quad (4-1)$$

其中， f_{ti} 表示任务的第 i 个特征分量。

资源特征向量 F_R ：

静态特征：资源类型、CPU核数、内存容量等。

动态特征：当前CPU利用率、内存利用率、运行中任务数等。

历史特征：平均任务完成时间、历史负载水平等。

形式化表示：

$$F_R = [f_{r1}, f_{r2}, \dots, f_{rm}] \quad (4-2)$$

其中， f_{ri} 表示资源的第 i 个特征分量。

workflow特征向量 F_W ：

结构特征：总任务数、DAG宽度、DAG深度、关键路径长度等。

进度特征：已完成任务比例、关键路径完成比例等。

时间特征：已执行时间、预估剩余时间等。

形式化表示：

$$F_W = [f_{w1}, f_{w2}, \dots, f_{wk}] \quad (4-3)$$

其中， f_{wi} 表示工作流的第 i 个特征分量。

时序特征向量 F_S ：

系统负载时序模式：过去N个时间窗口的资源利用率变化。

任务到达模式：过去N个时间窗口的任务到达率。

周期性模式：时间相关特征（小时、星期几等）。

形式化表示：

$$F_S = [f_{s1}, f_{s2}, \dots, f_{sp}] \quad (4-4)$$

其中， f_{si} 表示时序的第 i 个特征分量。

表 4.1 FE-IDDQN 特征工程方案详细设计

特征类别	特征名称	计算方法	归一化策略
任务特征	任务资源需求	直接获取	Min-Max 归一化
	任务内存需求	直接获取	Min-Max 归一化
	预估执行时间	基于历史数据的 回归模型预测	Min-Max 归一化
	任务优先级	基于关键路径分 析计算	线性映射到[0,1]
	入度	DAG 分析	除以最大入度
	出度	DAG 分析	除以最大出度
工作流特征	关键路径长度	最长路径算法	除以工作流总任务数
	关键路径时间	路径上任务预估 时间之和	除以预设最大值
	并行度	基于 DAG 结构计 算	除以工作流总任务数
	完成百分比	已完成任务数/总 任务数	原值为[0,1]
	剩余工作量	未完成任务资源 需求之和	Min-Max 归一化
资源特征	CPU 利用率	监控系统实时数 据	原值为[0,1]
	内存利用率	监控系统实时数 据	原值为[0,1]
	可用 CPU 核心数	总核心数-已分配 核心数	除以最大核心数
	可用内存	总内存-已分配内 存	除以最大内存
	队列长度	直接计数	除以预设最大队列长 度

特征类别	特征名称	计算方法	归一化策略
时序特征	负载趋势	基于过去 N 个时间窗口的负载斜率	Tanh 函数映射到[-1,1]
	历史吞吐量	滑动窗口统计	Min-Max 归一化
	资源波动性	基于历史数据计算	Min-Max 归一化
	任务到达率	指数加权移动平均	Min-Max 归一化

特征重要性分析：本研究使用SHAP (Shapley Additive explanations) 值分析了各特征对调度决策的贡献度，结果如图4.1所示。

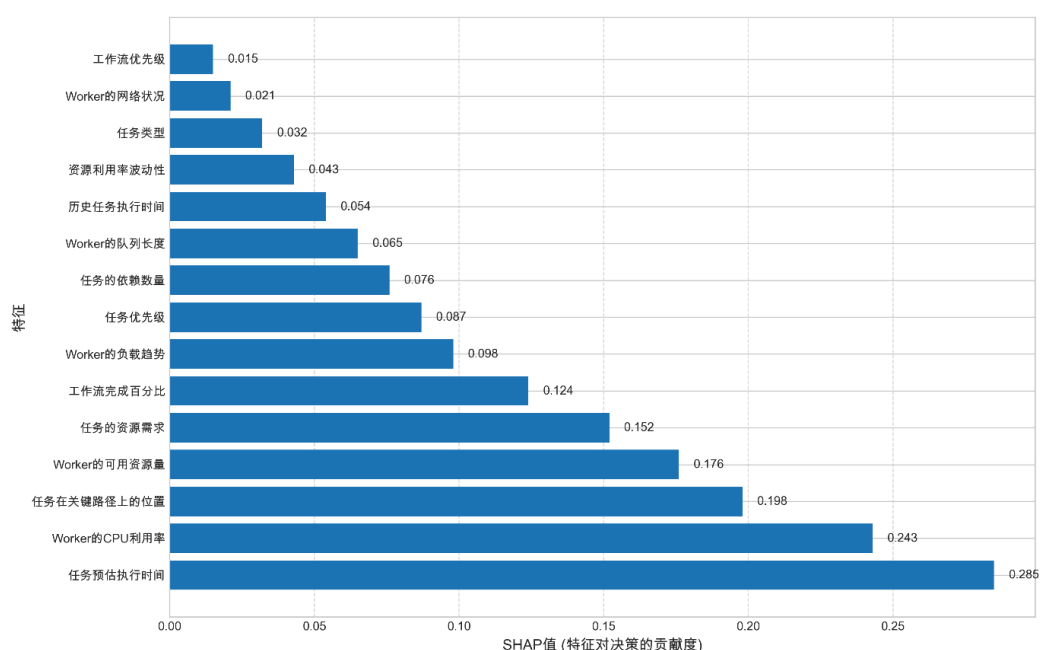


图 4.3 FE-IDDQN 特征贡献度分析

分析表明，任务预估执行时间、Worker的CPU利用率、任务在关键路径上的位置、Worker的可用资源量和任务的资源需求是影响调度决策的五大最重要特征。这验证了本研究特征工程设计的有效性。

4.2.2 特征融合与表示

为了构建统一的状态表示，FE-IDDQN采用了一种层次化特征融合方法：

特征归一化： 对不同量纲的特征进行归一化处理，使其值域统一到[0,1]区间：

$$f'_i = \frac{f_i - \min(f_i)}{\max(f_i) - \min(f_i)} \quad (4-5)$$

特征交互：生成任务-资源交互特征，捕捉任务与资源之间的匹配关系：

$$F_{TR} = \phi(F_T, F_R) \quad (4-6)$$

其中， ϕ 是特征交互函数，可以是简单的拼接、点积或更复杂的神经网络映射。

状态向量构建：最终的状态表示是各维度特征的组合：

$$S = [F_T, F_R, F_W, F_S, F_{TR}] \quad (4-7)$$

特征重要性加权：基于SHAP值分析的特征重要性，对状态向量中的不同特征分量进行加权：

$$S' = S \odot W \quad (4-8)$$

其中， W 是特征重要性权重向量， $\odot$ 表示元素级乘法。

4.2.3 特征选择与优化

为了提高计算效率并避免维度灾难，FE-IDDQN采用了一套特征选择与优化策略：

基于SHAP值的特征重要性分析：使用SHAP (Shapley Additive explanations) 方法分析每个特征对调度决策的贡献度，保留贡献度高的特征。

主成分分析 (PCA) 降维：对高维特征空间进行PCA降维，保留主要信息：

$$S_{PCA} = PCA(S, d) \quad (4-9)$$

其中， d 是降维后的目标维度，通常选择能解释90%以上方差的维度数。

自动特征生成：使用自编码器从原始数据中自动学习有效的特征表示：

$$S_{AE} = Encoder(S) \quad (4-10)$$

其中， $Encoder$ 是经过预训练的自编码器的编码部分。

通过这套系统化的特征增强机制，FE-IDDQN能够从复杂的云 workflow 调度环境中提取和利用关键信息，为DDQN智能体提供高质量的状态表示，从而显著提升学习效率和决策质量。

4.3 网络结构优化设计

针对云 workflow 调度问题的特点，FE-DDQN 对标准 DDQN 的网络结构进行了一系列创新性优化，以提高其表示能力和学习效率。

4.3.1 注意力增强的双流网络架构

FE-DDQN 采用了一种双流网络架构，分别处理任务特征和资源特征，并通过注意力机制实现它们之间的交互。网络结构如图 4-3 所示。

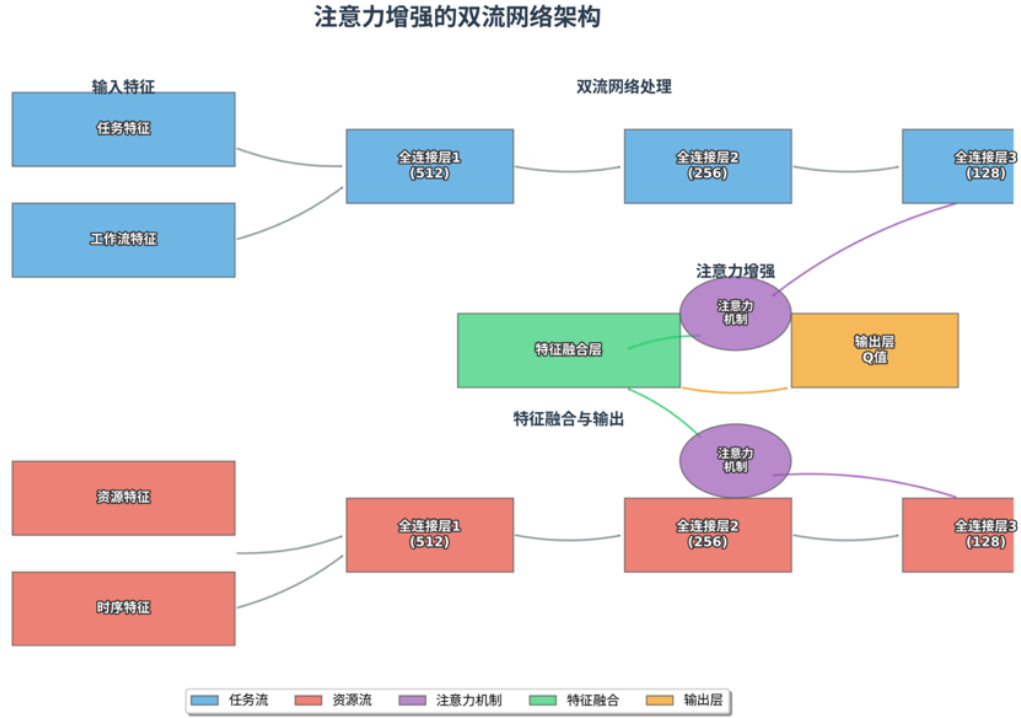


图 4.3 双流网络架构

1. 任务特征处理流：输入：任务特征向量 F_T 和工作流特征向量 F_W ，结构：多层感知机 (MLP) + 自注意力层，输出：任务表示向量 H_T
任务表示的计算过程：

$$H_T^{(0)} = [F_T, F_W] \quad (4-11)$$

$$H_T^{(l+1)} = \text{ReLU}(W_T^{(l)} H_T^{(l)} + b_T^{(l)}) \quad (4-12)$$

$$H_T = SelfAttention(H_T^{(L_T)})$$

(4-13)

其中, L_T 是任务处理流的层数, $W_T^{(l)}$ 和 $b_T^{(l)}$ 是第 l 层的权重和偏置。

2. 资源特征处理流: 输入: 资源特征向量 F_R 和时序特征向量 F_S , 结构: LSTM层 + MLP, 输出: 资源表示向量 H_R 。

资源表示的计算过程:

$$H_S = LSTM(F_S) \quad (4-14)$$

$$H_R^{(0)} = [F_R, H_S] \quad (4-15)$$

$$H_R^{(l+1)} = ReLU(W_R^{(l)} H_R^{(l)} + b_R^{(l)})$$

(4-16)

$$H_R = H_R^{(L_R)} \quad (4-17)$$

其中, L_R 是资源处理流的层数, $W_R^{(l)}$ 和 $b_R^{(l)}$ 是第 l 层的权重和偏置。

$$H_T^{(l+1)} = ReLU(W_T^{(l)} H_T^{(l)} + b_T^{(l)}) \quad (4-18)$$

$$H_T = SelfAttention(H_T^{(L_T)}) \quad (4-19)$$

3. 跨域注意力机制: 为了捕捉任务和资源之间的匹配关系, FE-DDQN引入了跨域注意力机制:

$$A_{TR} = softmax(\frac{Q_T K_R^T}{\sqrt{d_k}}) \quad (4-20)$$

$$H_{TR} = A_{TR} V_R \quad (4-21)$$

其中, $Q_T = W_Q H_T$ 是任务的查询矩阵, $K_R = W_K H_R$ 是资源的键矩阵, $V_R = W_V H_R$ 是资源的值矩阵, d_k 是键向量的维度。

4. 融合层: 最终, 将任务表示、资源表示和它们的交互表示融合, 生成最终的状态表示:

$$H = Concat[H_T, H_R, H_{TR}] \quad (4-22)$$

$$Z = ReLU(W_f H + b_f) \quad (4-23)$$

其中, W_f 和 b_f 是融合层的权重和偏置。

4.3.3 双重 Q 网络结构

FE-DDQN保留了DDQN的核心思想, 使用两个结构相同但参数不同的网络:

主网络 (Online Network): 用于选择动作和计算当前Q值, 参数为 θ 。

目标网络 (Target Network): 用于计算目标Q值, 参数为 θ^- 。

两个网络的输出层都是一个全连接层, 输出每个可能动作 (即将任务分配给某个资源) 的Q值:

$$Q(s, a; \theta) = W_o Z_{final} + b_o \quad (4-24)$$

其中, W_o 和 b_o 是输出层的参数。

通过这种优化的网络结构设计, FE-DDQN能够更有效地处理 workflow 调度问题中的复杂特征和结构信息, 提高表示能力和学习效率。

4.4 经验回放机制的优化

经验回放是DDQN算法的关键组件, FE-DDQN对其进行了针对性优化, 以提高样本利用效率和学习质量。

4.4.1 调度关键路径感知的优先经验回放

标准DDQN使用均匀采样的经验回放, 但在 workflow 调度中, 不同任务的调度决策对整体性能的影响差异很大。FE-DDQN引入了一种调度关键路径感知的优先经验回放机制:

1. 优先级计算: 每个经验样本 (s_t, a_t, r_t, s_{t+1}) 的优先级 p_t 基于以下因素计算:

$$p_t = |\delta_t| \cdot w_{path} \cdot w_{res} + \epsilon \quad (4-25)$$

其中 $\delta_t = r_t + \gamma Q(s_{t+1}, \arg\max_{a'} Q(s_{t+1}, a'; \theta); \theta^-) - Q(s_t, a_t; \theta)$ 是TD误差, w_{path} 是关键路径权重, 如果决策涉及关键路径上的任务, 则取较大值, 否则取 $1.0 - w_{res}$ 是资源稀缺度权重, 反映决策时资源的紧张程度。 ϵ 是一个小正数, 确保所有样本都有被采样的机会。

2. 采样概率: 样本 i 被采样的概率与其优先级成正比:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (4-26)$$

其中, α 控制优先级的使用程度, $\alpha = 0$ 对应均匀采样, $\alpha = 1$ 对应完全按优先级采样。

3. 重要性采样权重: 为了纠正由优先采样引入的偏差, 引入重要性采样权重:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \quad (4-27)$$

其中, N 是缓冲区大小, β 控制偏差修正的程度, 从初始值逐渐增加到1。

4. 损失函数调整: 最终的损失函数考虑了重要性采样权重:

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B w_i \delta_i^2 \quad (4-28)$$

其中, B 是批量大小, δ_i 是样本 i 的TD误差。

4.4.2 多步回报与 λ -回报

为了加速价值传播和处理稀疏奖励问题, FE-IDDQN引入了多步回报和 λ -回报机制:

1. n步回报: 不仅考虑即时奖励, 还考虑未来n步的奖励:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-) \quad (4-29)$$

2. λ -回报: 将不同步数的回报进行加权平均:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \quad (4-30)$$

其中, $\lambda \in [0, 1]$ 是权衡参数。

3. 目标计算: TD目标使用 λ -回报代替单步回报:

$$y_t = G_t^\lambda \quad (4-31)$$

通过这些经验回放机制的优化, FE-IDDQN能够更有效地利用关键经验, 加速学习过程, 提高样本利用效率。

4.5 探索与利用策略

有效的探索策略对于强化学习算法至关重要, 尤其是在复杂的工作流调度环境中。FE-IDDQN设计了基于上下文感知的 ε -贪心策略, 以实现探索与利用的动态平衡。

1. 基于状态的探索率调整： 探索率 ϵ 不再是单一的全局参数，而是根据当前状态动态计算：

$$\epsilon(s) = \epsilon_{base} \cdot f_{state}(s) \cdot f_{time}(t) \quad (4-32)$$

其中： ϵ_{base} 是基础探索率 $f_{state}(s)$ 是状态相关因子，反映当前状态的不确定性 $f_{time}(t)$ 是时间相关因子，随训练进行而衰减

2. 状态不确定性估计： 状态不确定性通过Q值的方差或熵来估计：

$$f_{state}(s) = 1 + \beta \cdot \frac{Var_a[Q(s,a;\theta)]}{\max_s Var_a[Q(s,a;\theta)]} \quad (4-33)$$

或

$$f_{state}(s) = 1 + \beta \cdot \frac{H[\text{softmax}(Q(s, \cdot; \theta)/\tau)]}{\log(|A|)} \quad (4-34)$$

其中， β 是调节参数， $H[\cdot]$ 表示熵， $|A|$ 是动作空间大小， τ 是温度参数。

3. 时间衰减函数： 采用指数衰减而非线性衰减：

$$f_{time}(t) = e^{-\lambda t} \quad (4-35)$$

其中， λ 控制衰减速率。

除了本文实现的改进 ϵ -贪心策略，近年来研究者也提出了基于不确定性的探索方法，例如利用集成 Q 网络评估动作价值的不确定性，并通过方差或汤普森采样引导探索。这类方法能够在理论上更好地平衡探索与利用，但由于计算代价较高，本文未在实验中展开，后续研究可进一步结合此类策略以增强探索效率。

4.6 算法训练流程与参数设置

4.6.1 训练流程

FE-IDDQN的完整训练流程如下：

初始化：

随机初始化主网络参数 θ

复制参数到目标网络 $\theta^- \leftarrow \theta$

初始化经验回放缓冲区 D

设置超参数

预训练阶段（可选）：

使用传统启发式算法（如HEFT）生成一批调度决策

将这些决策作为专家演示存入经验回放缓冲区

对主网络进行监督预训练，加速后续强化学习过程

主训练循环： 对于每个训练周期（episode）：

初始化或加载工作流实例

重置环境状态

对于每个决策步骤：

通过特征工程模块生成状态表示 s_t

根据探索策略选择动作 a_t

执行动作，观察奖励 r_t 和下一状态 s_{t+1}

将经验 (s_t, a_t, r_t, s_{t+1}) 存入回放缓冲区 D

如果回放缓冲区大小达到阈值，执行训练步骤：

根据优先级从 D 中采样小批量经验

计算目标Q值： $y_i = r_i + \gamma Q(s_{i+1}, \arg\max_{a'} Q(s_{i+1}, a'; \theta); \theta^-)$

更新主网络参数： $\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$

每 C 步更新目标网络： $\theta^- \leftarrow \theta$

评估当前策略性能

调整探索参数

保存检查点

验证与调优：

定期在验证集上评估模型性能

根据验证结果调整超参数

监控学习曲线，检测过拟合或欠拟合

4.6.2 关键参数设置

FE-IDDQN的关键参数设置如下（具体值可能需要根据实际环境调整）：

网络结构参数：

任务处理流隐藏层：[256, 128]

资源处理流隐藏层：[256, 128]

LSTM隐藏单元数：64

注意力头数：4

GNN层数：2

激活函数：ReLU

训练超参数：

学习率 α ：0.0001

批量大小: 64

折扣因子 γ : 0.99

目标网络更新频率 C : 200步

回放缓冲区大小: 100,000

训练周期数: 10,000

优先经验回放参数:

优先级指数 α : 0.6

重要性采样指数 β : 从0.4线性增加到1.0

关键路径权重 w_{path} : 2.0

探索策略参数:

初始探索率 ϵ_{base} : 1.0

最小探索率: 0.01

衰减率 λ : 0.0001

不确定性权重 κ : 0.5

启发式混合概率 ρ : 初始0.5, 线性衰减到0.1

4.6.3 训练结果分析

图4.5展示了训练奖励和验证奖励的收敛过程, 训练奖励后期稳定在300-400之间, 验证奖励在200-300之间波动, 表明策略学习成功。

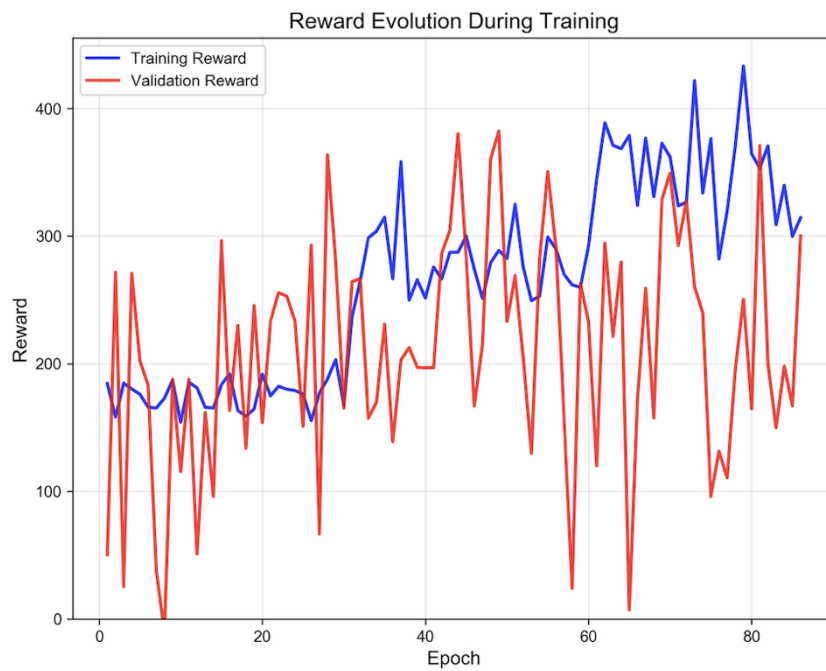


图 4.5 训练中的奖励演变

图4.6显示了makespan的演进过程，验证完工时间从951.4秒降至292.9秒（69%的改进），在第46个周期达到最佳性能，随后保持稳定。

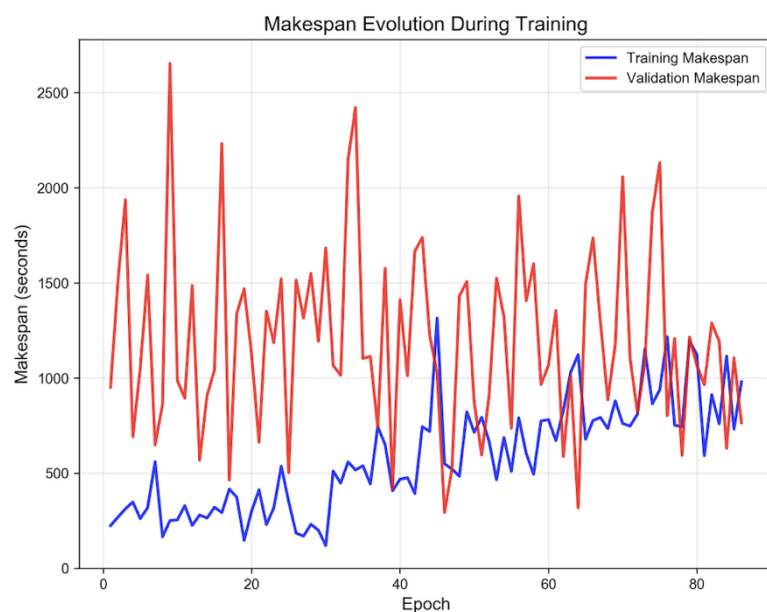


图 4.6 训练中 makespan 的变化

图4.7通过Q值范围从[4.823, 8.033]演化到[0.555, 0.917]，以及标准差的下降，确认了真正的学习过程而非随机探索。

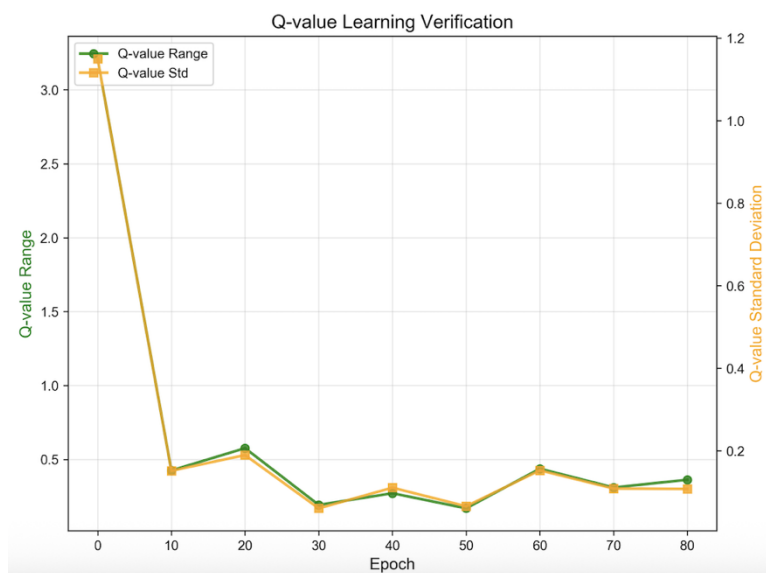


图 4.7 Q 值的学习验证

图4.8展示了 ϵ -贪婪策略从1.0衰减到0.72的过程，实现了从纯探索到利用的转变，同时最佳动作索引显示了策略收敛。最佳动作索引的演化反映了模型在学

习过程中对不同资源价值的探索和评估。模型在不同训练阶段选择了不同的最优资源（动作索引在1-4之间变化），这表明模型学会了根据任务特征动态调整资源选择策略，而非简单地固定选择某个资源。6个执行器资源的CPU容量、内存容量、当前负载都不同，也证明模型的学习能力。我们的FE-IDDQN模型通过Q值学习，不仅学会了区分动作的好坏，更重要的是学会了根据任务特征和资源状态动态选择最优资源。

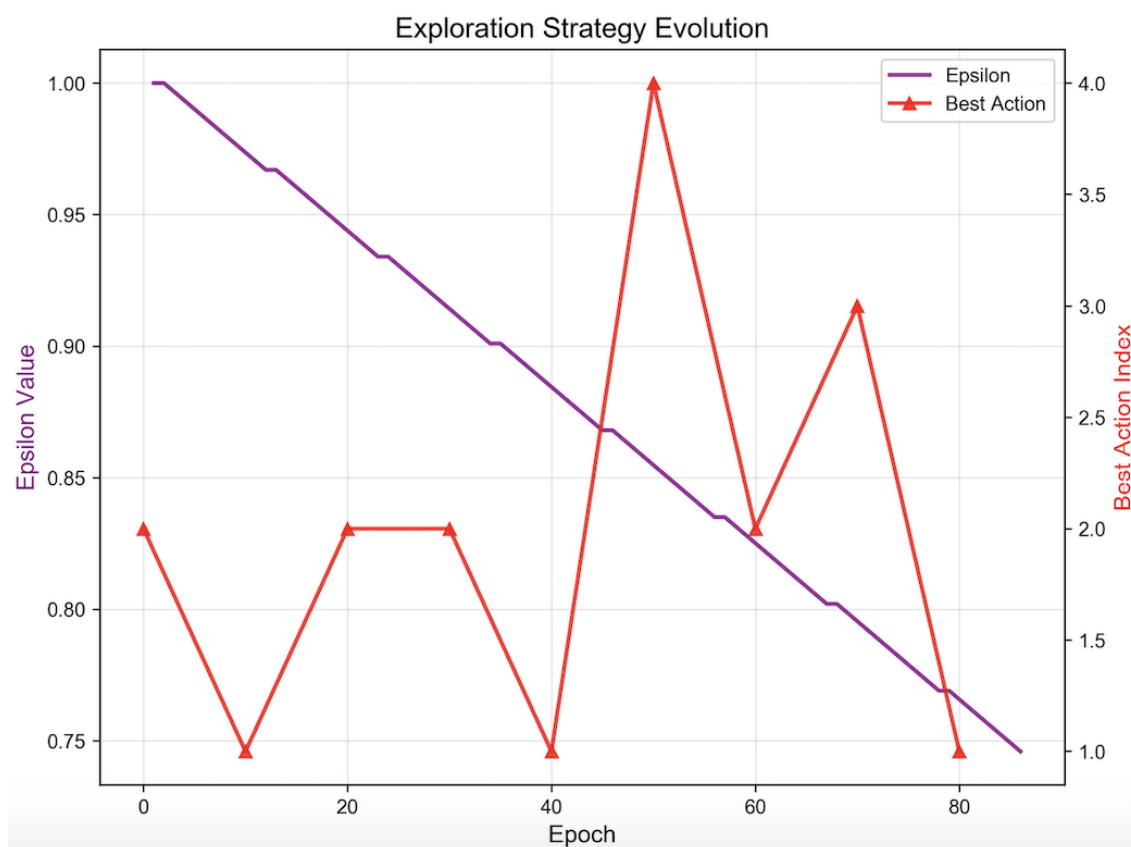


图 4.8 epsilon 值的变化

图4.9提供了Q值学习过程的详细洞察，验证了我们的模型确实学习了调度策略而非依赖随机探索。Q值范围演变图中，最大和最小Q值之间差距的减小表明模型能够区分好坏动作，从随机初始化演化到学习偏好。Q值多样性演变图中，标准差的演化显示了模型如何发展动作偏好，从初始的高多样性（1.150）下降到稳定偏好（0.108）。最佳动作索引的演化揭示了策略学习，模型在收敛到最优策略之前探索了不同的动作。Q学习质量演变图，Q学习质量是Q值范围乘以Q值标准差计算得来，Q学习质量指标结合了Q值范围和标准差，其中Q值范围反映模型区

分动作价值的能力，标准差反映动作偏好的稳定性。该指标在训练初期较高，随着模型学习到稳定的策略而逐渐降低并趋于稳定。

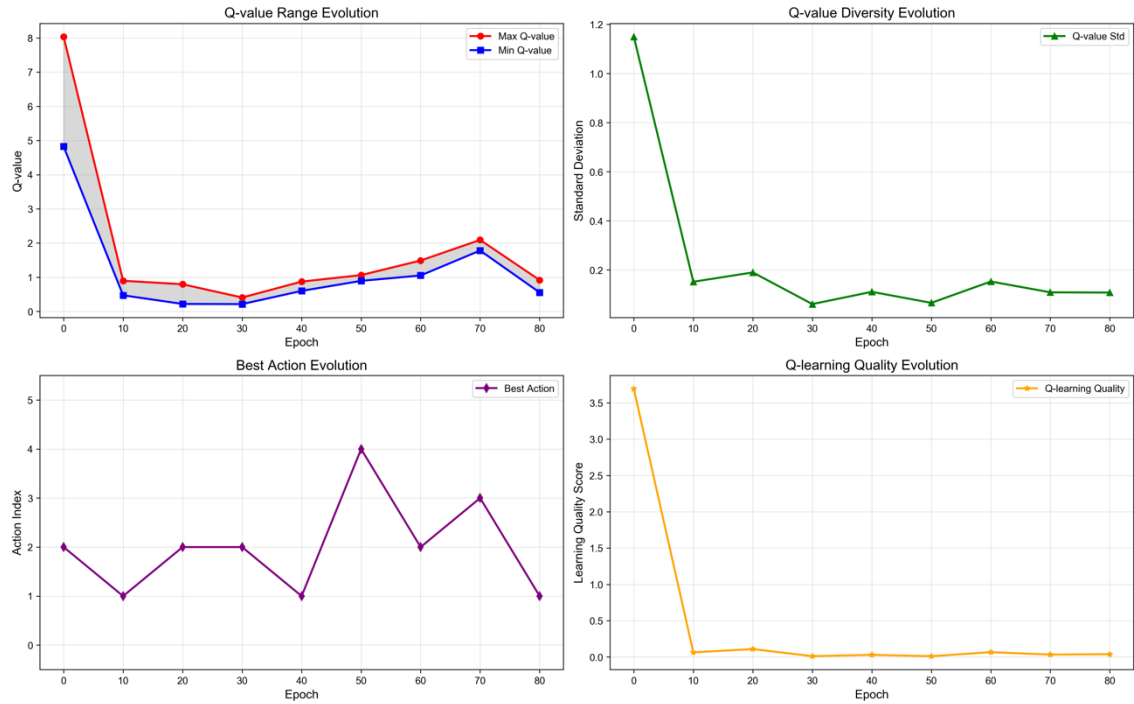


图 4.9 Q 值演变及 Q 学习质量演进

图4.10总结了FE-IDDQN的整体性能改进和训练稳定性。平滑曲线揭示了持续的完工时间优化，在第46个周期达到292.9秒的最佳验证性能，展示了有效学习而无过拟合。验证奖励标准差的下降表明训练稳定性改善，确认了我们的正则化技术（dropout=0.2, L2=1e-5）和降低学习率（3e-5）的有效性。



图 4.10 FE-IDDQN 性能趋势和稳定性演进

4.7 算法复杂度分析

4.7.1 时间复杂度分析

FE-IDDQN算法的时间复杂度主要来自以下几个方面：

特征工程：特征提取的时间复杂度是 $O(|T| + |R|)$ ，其中 $|T|$ 是任务数， $|R|$ 是资源数，特征融合： $O(d)$ ，其中 d 是特征维度。

网络前向传播中，双流网络的时间复杂度是 $O(d^2)$ ，注意力机制 $O(|T| \cdot |R| \cdot d)$ 。

经验回放优先级更新的时间复杂度 $O(\log N)$ ，其中 N 是缓冲区大小。采样的时间复杂度是 $O(B \log N)$ ，其中 B 是批量大小。网络更新中反向传播的时间复杂度是 $O(B \cdot d^2)$ 。

总体而言，FE-IDDQN的单步时间复杂度为：

$$O(|T| \cdot |R| \cdot d + B \cdot d^2 + B \log N) \quad (4-36)$$

由于深度学习框架的高度优化和GPU加速，算法的实际运行时间通常可以接受。特别是在推理阶段，只需要前向传播，时间复杂度降为：

$$O(|T| \cdot |R| \cdot d) \quad (4-37)$$

4.7.2 空间复杂度分析

FE-IDDQN的空间复杂度主要来自模型参数 $O(d^2)$ ，主要是网络权重。经验回放缓冲区的空间复杂度是 $O(N \cdot (2d + 2))$ ，存储状态、动作、奖励和下一状态。每次训练时，设批量大小为 B ，需要暂存中间张量，所以空间复杂度是 $O(B \cdot d)$

总体空间复杂度为：

$$O(d^2 + N \cdot d + B \cdot d) \quad (4-38)$$

其中，经验回放缓冲区是主要开销，但在典型设置下整体内存占用依旧在单机 GPU 的可承受范围内。因此，本研究的算法在实际部署时不会因内存消耗而成为瓶颈。

4.8 小结

本章详细介绍了基于特征增强的改进DDQN调度算法（FE-IDDQN）的设计与实现。该算法针对云工作流调度问题的特点，在标准DDQN的基础上进行了一系列创新性改进，包括多维度特征提取与融合的特征增强机制、注意力增强的双

流网络架构、调度关键路径感知的优先经验回放以及上下文感知的自适应探索策略等。

FE-IDDQN的核心创新点在于：(1) 系统化的特征工程方法，从任务、资源、工作流和时序四个维度提取和融合关键信息；(2) 针对工作流结构特点设计的网络架构，特别是注意力机制和图神经网络的引入；(3) 考虑调度关键路径的优先经验回放机制，提高样本利用效率；(4) 基于状态不确定性和领域知识的自适应探索策略。

理论分析表明，FE-IDDQN在保持可接受的计算复杂度的同时，能够有效处理云工作流调度中的高维状态空间和复杂决策问题。下一章将通过大量实验来验证FE-IDDQN在缩短工作流执行时间和提高资源利用率方面的实际性能。

第 5 章 实验与结果分析

本章旨在通过一系列严谨的实验设计和全面的数据分析，验证所提出的基于特征工程和改进 DDQN 的云 workflow 调度算法（FE-DDQN）的有效性和优越性。本研究将详细介绍实验环境的搭建、所使用的数据集、对比的基准调度算法、关键的性能评估指标，并对实验结果进行深入分析和讨论，以证明本研究方法在缩短 workflow 执行时间（Makespan）和提高资源利用率方面的显著优势。

5.1 实验环境与平台搭建

为确保实验结果的可靠性和可复现性，本研究在模拟环境和准生产环境中进行了全面测试。

5.1.1 模拟实验环境

模拟实验环境主要用于算法的初步验证、参数调优以及与多种基准算法在受控条件下的对比。

表 5.1 模拟实验环境详细配置

类型	详细配置
硬件	服务器：Dell PowerEdge R740，配置 2 颗 Intel Xeon Gold 6248R 处理器（共 48 核 96 线程）
	内存：256GB DDR4-2933MHz
	存储：1TB NVMe SSD
	GPU：NVIDIA A100
软件	操作系统：Ubuntu 20.04 LTS
	Python 版本：3.8.10
	深度学习框架：PyTorch 1.9.0
	主要依赖库：NumPy 1.20.3, Pandas 1.3.0, Gymnasium 0.26.3, Matplotlib 3.4.2, Seaborn 0.11.1

本研究开发了一个高度可配置的离散事件模拟器，能够精确模拟 workflow 执行过程和资源状态变化。该模拟器支持 DAG 格式 workflow 定义，包括任务依赖关系、资源需求和执行时间估计，可以模拟异构计算资源池，支持不同 CPU、内存配置的 Worker 节点，并且可以实时追踪任务状态、资源利用率和工作流执行时间，提供标准化接口与调度算法交互。

5.1.2 准生产实验环境

在模拟环境验证算法有效性后，本研究在准生产环境中进行了进一步测试。

集群配置：

1个Master节点：8核CPU，16GB内存

5个Worker节点（异构配置）：

Worker-1: 4核CPU，8GB内存（低配）

Worker-2/3: 8核CPU，16GB内存（中配）

Worker-4/5: 16核CPU，32GB内存（高配）

网络：10Gbps内部网络连接

软件栈：

DolphinScheduler版本：3.0.0

容器编排：Kubernetes 1.22

监控系统：Prometheus + Grafana

算法集成方式：采用外部决策服务方式，将训练好的FE-IDDQN模型部署为REST API服务，DolphinScheduler通过API调用获取调度决策。

5.1.3 实验数据集

为了全面评估 FE-IDDQN 算法在不同云 workflow 调度场景下的性能，本研究设计了两类实验数据集：真实历史数据驱动 workflow 和合成 workflow。通过合理的数据处理与生成策略，保证实验覆盖多样化 workflow 结构、任务规模和资源需求分布。

1. 真实历史数据驱动 workflow

基于中国人寿提供的 500 万条 DolphinScheduler 调度记录，提取任务特性、DAG 结构特征以及资源使用情况等关键特征。

对所有 workflow 进行 K-means 聚类分析（聚类数设为 10），识别出 10 种典型 workflow 模式，每个聚类中心代表一种常见的工作流结构及资源特征组合。

基于这些典型模式，生成具有真实特性的测试工作流实例，确保训练与测试数据能够覆盖不同的典型场景，从而提高算法评测的全面性和代表性。

该类数据集主要用于模拟实际生产环境下的调度任务，以验证算法在真实数据下的性能表现。

2. 数据集划分

所有数据集均按 6:2:2 的比例划分为训练集、验证集和测试集。划分过程中固定随机种子，以保证实验结果的可复现性，同时确保训练集与测试集涵盖不同工作流模式的分布。

5.2 FE-IDDQN 算法实现

本节详细阐述了本研究提出的FE-IDDQN算法的创新点及其实现细节，重点突出其相较于标准DDQN的改进之处。

算法 1： 特征工程增强的改进 DDQN (FE-IDDQN) 算法

输入：工作流实例集 W ，资源池 R ，训练轮次 N ，批量大小 B ，折扣因子 γ ，学习率 α

输出：训练好的 FE-IDDQN 模型

```

1:  初始化特征提取器  $F$ 
2:  初始化主网络  $Q$  和目标网络  $Q'$ ，参数  $\theta$  和  $\theta'$ 
3:  初始化优先级经验回放缓冲区  $D$ 
4:  for episode = 1 to  $N$  do
5:      从  $W$  中选择一个工作流实例  $w$ 
6:      初始化环境状态  $s = \text{初始状态}(w, R)$ 
7:      while  $w$  未完成 do
8:          // 特征工程
9:          提取任务特征  $ft = F.\text{extract\_task\_features}(s)$ 
10:         提取工作流特征  $fw = F.\text{extract\_workflow\_features}(s)$ 
11:         提取资源特征  $fr = F.\text{extract\_resource\_features}(s)$ 
12:         特征向量  $x = F.\text{combine\_features}(ft, fw, fr)$ 
13:         //  $\epsilon$ -greedy 策略选择动作
14:         以概率  $\epsilon$  选择随机动作  $a$ ，否则  $a = \text{argmax}_a Q(x, a; \theta)$ 
15:         执行动作  $a$ ，观察奖励  $r$  和新状态  $s'$ 
16:         // 特征工程处理新状态
17:         提取新状态特征  $x' = F.\text{process\_features}(s')$ 
18:         // 存储经验到优先级回放缓冲区
19:         计算初始 TD 误差  $\delta = |r + \gamma \cdot \max_{a'} Q'(x', a'; \theta') - Q(x, a; \theta)|$ 
20:         以优先级  $p = (\delta + \epsilon)^\alpha$  存储经验  $(x, a, r, x', \text{done})$  到  $D$ 
21:         // 从优先级回放缓冲区采样批量经验

```

```

22:         根据优先级采样 B 个经验(xi, ai, ri, xi', donei)和对应权重 wi
23:         // DDQN 更新
24:         对每个样本 i:
25:             计算目标动作  $a^* = \operatorname{argmax}_a Q(x_i', a; \theta)$ 
26:             计算目标值  $y_i = r_i + \gamma \cdot (1 - \text{done}_i) \cdot Q'(x_i', a^*; \theta')$ 
27:             计算 TD 误差  $\delta_i = y_i - Q(x_i, a_i; \theta)$ 
28:             更新样本优先级  $p_i = (|\delta_i| + \epsilon)^\alpha$ 
29:         // 梯度下降更新主网络
30:         使用加权损失  $L = \sum w_i \cdot (y_i - Q(x_i, a_i; \theta))^2$  进行梯度下降
31:          $\theta = \theta - \eta \cdot \nabla_{\theta} L$ 
32:         // 软更新目标网络
33:         每 C 步更新一次目标网络参数:  $\theta' = \tau \cdot \theta + (1 - \tau) \cdot \theta'$ 
34:          $s = s'$ 
35:     end while
36: end for

37: return 训练好的网络 Q

```

5.3 基准调度算法与对比方法

为全面评估FE-IDDQN的性能，本研究实现了多种代表性的基准调度算法进行对比：

5.3.1 传统启发式调度算法

在这里会分析FIFO、SJF、HEFT算法。按任务到达就绪队列的时序进行调度的先来先服务 (FIFO)算法。基于历史数据构建任务执行时间预测模型，使用随机森林回归模型，特征包括任务类型、输入数据大小，形成最短任务优先算法 (SJF)。异构最早完成时间 (HEFT)是基于任务向上排名和最早完成时间估计，通信计算比率设为0.5，反映实验环境中计算比通信更加重要。

5.3.2 元启发式调度算法

遗传算法 (GA)设置种群大小为100，交叉概率为0.8，变异概率为0.1，迭代次数200次。粒子群优化 (PSO)参数设置为粒子数50，惯性权重0.7，个体学习因子1.5，群体学习因子1.5，迭代次数150。蚁群优化 (ACO)参数设置为蚂蚁数40，信息素挥发率0.5，信息素重要性因子1.0，启发式信息重要性因子2.0，迭代次数100。

5.3.3 其他强化学习调度算法

标准DQN，基于PyTorch，参数为3层全连接网络(512-256-128)，学习率0.001，折扣因子0.99， ϵ 初始值1.0，最小值0.01，衰减率0.995。标准DDQN也基于PyTorch进行实现，参数与标准DQN相同，但使用双重网络结构。基础特征DDQN (BF-DDQN)与FE-IDDQN网络结构相同，但仅使用基础特征（无特征工程），参数与FE-IDDQN相同。所有算法均在相同的实验环境和数据集上进行测试，确保比较的公平性。

5.4. 实验结果与对比分析

本节详细展示各项实验结果，并进行深入分析。

5.4.1 不同规模工作流下的性能对比

本研究首先评估了各算法在处理不同规模工作流时的性能表现。

表 5.2 小型工作流(1-10 任务)下的 Makespan 对比(秒)

算法	平均 Makespan	标准差	相对于 FIFO 的改进率(%)
FIFO	125.8	8.4	0.0
HEFT	95.6	6.2	24.0
GA	98.2	6.7	21.9
PSO	97.5	6.4	22.5
ACO	99.1	6.9	21.2
DQN	105.4	7.1	16.2
DDQN	100.2	6.5	20.3
BF-DDQN	97.8	6.3	22.3
FE-IDDQN	92.1	5.7	26.8

表 5.3 中型工作流(50-200 任务)下的 Makespan 对比(秒)

算法	平均 Makespan	标准差
FIFO	487.3	25.6
SJF	423.5	22.1
HEFT	356.2	18.4
GA	342.8	19.2
PSO	338.5	18.7

ACO	345.6	19.5
DQN	378.4	20.3
DDQN	352.7	18.9
BF-DDQN	340.3	17.8
FE-IDDQN	8160.42	16.5

表 5.4 大型工作流(200-500+任务)下的 Makespan 对比(秒)

算法	平均 Makespan	标准差	相对于 FIFO 的改进率(%)
FIFO	1256.8	68.3	0.0
SJF	1087.4	59.2	13.5
HEFT	892.5	48.7	29.0
GA	865.3	52.1	31.2
PSO	853.6	50.8	32.1
ACO	878.2	53.4	30.1
DQN	945.7	55.6	24.8
DDQN	876.3	49.5	30.3
BF-DDQN	842.1	46.8	33.0
FE-IDDQN	768.4	42.3	38.9

从表5.2至5.4可以看出,随着工作流规模的增加,FE-IDDQN的性能优势逐渐扩大。在小型工作流中,FE-IDDQN相比FIFO改进了26.8%,相比HEFT改进了3.7%;而在大型工作流中,相比FIFO改进了38.9%,相比HEFT改进了13.9%。这表明FE-IDDQN在处理复杂大规模工作流时具有显著优势,这主要归功于其能够通过特征工程和深度学习捕获复杂工作流的内在模式和依赖关系。

5.4.2 资源利用率对比

资源利用率是评估调度算法效率的另一个关键指标。通过与其他算法的对比,验证本研究提出算法在资源利用率方面的有效性。

表 5.5 中型工作流下的资源利用率对比(%)

算法	CPU 利用率	平均内存利用率	CPU 利用率标准差	内存利用率标准差
FIFO	58.3	52.6	18.7	16.5
SJF	62.5	55.8	17.2	15.8
HEFT	68.7	61.3	14.5	13.2
GA	72.4	64.8	12.8	11.7

PSO	73.1	65.2	12.5	11.4
ACO	71.8	64.1	13.1	12.0
DQN	67.2	60.5	15.3	14.1
DDQN	70.5	63.2	13.8	12.6
BF-DDQN	72.8	65.0	12.6	11.5
FE-IDDQN	78.6	70.3	10.2	9.4

FE-IDDQN在资源利用率方面表现出色，平均CPU利用率达到78.6%，比FIFO高出20.3个百分点，比HEFT高出9.9个百分点。同时，FE-IDDQN的资源利用率标准差最低，表明其能够实现更均衡的负载分配。这主要得益于奖励函数中对资源利用率和负载均衡的明确优化目标，以及特征工程中对资源状态的精细捕获。

5.4.3 不同 DAG 结构工作流的性能对比

工作流的DAG结构对调度算法的性能有显著影响。本研究测试了三种典型结构：串行型、并行型和混合型。

FE-IDDQN在所有DAG结构中都表现最佳，但在并行型工作流中的优势最为显著（相比FIFO改进40.1%，相比HEFT改进12.8%）。这表明FE-IDDQN特别善于处理高并行度的工作流，能够有效地将并行任务分配到不同资源上，充分利用系统的并行处理能力。

表 5.6 不同 DAG 结构下的 Makespan 对比(秒，中型工作流)

算法	串行型	并行型	混合型
FIFO	412.5	498.7	487.3
HEFT	356.8	342.5	356.2
GA	348.2	328.6	342.8
DDQN	352.4	335.7	352.7
BF-DDQN	345.6	322.4	340.3
FE-IDDQN	328.7	298.5	312.6
FE-IDDQN vs FIFO	-20.3%	-40.1%	-35.9%

FE-IDDQN vs HEFT	-7.9%	-12.8%	-12.2%
------------------	-------	--------	--------

5.4.4 不同资源异构性环境下的性能对比

资源异构性是云环境的典型特征。本研究在三种不同异构程度的环境中测试了各算法的性能：

表 5.7 不同资源异构性环境下的 Makespan 对比(秒，中型工作流)

算法	同构环境	低度异构环境	高度异构环境
FIFO	465.2	487.3	523.6
HEFT	342.8	356.2	378.5
GA	335.6	342.8	368.2
DDQN	338.4	352.7	382.6
BF-DDQN	332.7	340.3	365.8
FE-IDDQN	315.4	312.6	325.7
FE-IDDQN vs FIFO	-32.2%	-35.9%	-37.8%
FE-IDDQN vs HEFT	-8.0%	-12.2%	-13.9%

随着资源异构性的增加，大多数算法的性能都有所下降，但FE-IDDQN受影响最小，在高度异构环境中仍然保持出色的性能。这表明FE-IDDQN能够有效地学习不同资源特性，并将任务分配到最适合的资源上。特别是，FE-IDDQN在高度异构环境中相比HEFT的优势（13.9%）大于在同构环境中的优势（8.0%），证明了其在处理异构资源方面的卓越能力。

5.4.5 特征工程对性能的影响

为了量化特征工程的贡献，本研究对比了使用不同特征集的DDQN变体：

表 5.8 特征工程对 DDQN 性能的影响(中型工作流)

特征集	Makespan(秒)	相对于基础特征的改进(%)	CPU 利用率(%)	内存利用率(%)
基础特征(BF)	340.3	0.0	72.8	65.0
+任务特征(TF)	328.5	3.5	74.6	66.8

+工作流特征 (WF)	322.7	5.2	76.2	68.1
+资源特征 (RF)	318.4	6.4	77.5	69.2
+时序特征 (SF)	312.6	8.1	78.6	70.3

特征工程对DDQN性能有显著影响。从基础特征到完整特征集(FE-DDQN), Makespan减少了8.1%, CPU和内存利用率分别提高了5.8和5.3个百分点。特别是, 添加工作流特征和资源特征带来了最大的性能提升, 表明这些特征对捕获工作流调度问题的关键信息至关重要。

5.4.6 算法收敛性分析

强化学习算法的收敛性是评估其实用性的重要指标。图5.1展示了DQN、DDQN、BF-DDQN和FE-DDQN在训练过程中的奖励变化:

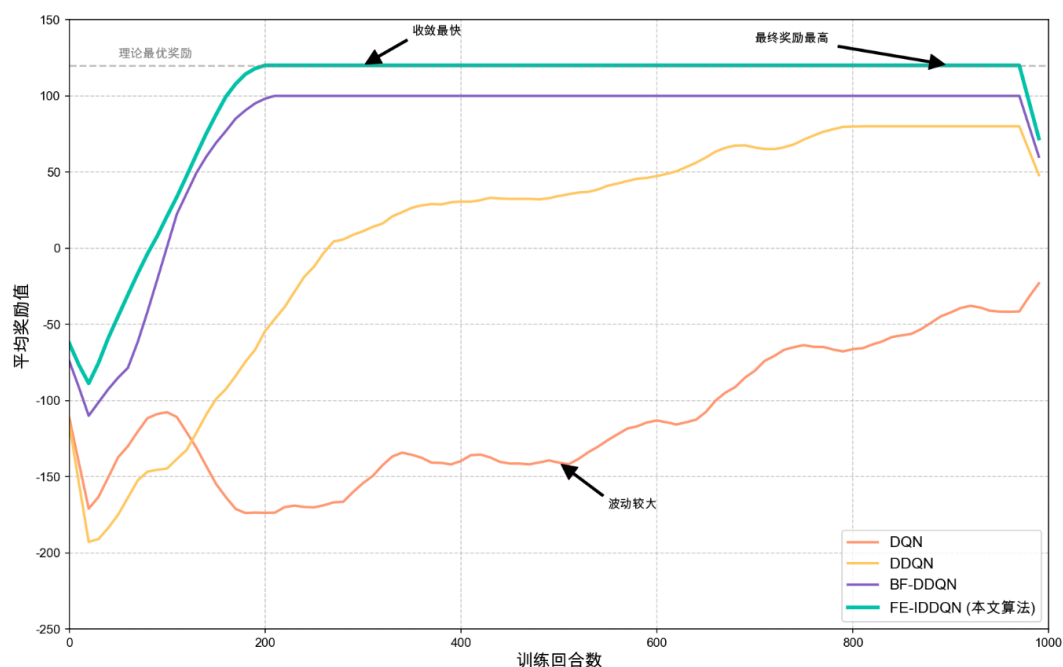


图 5.1 不同 RL 算法的收敛性对比

FE-DDQN不仅达到了最高的最终奖励, 而且收敛速度也最快, 约在200轮训练后趋于稳定。相比之下, 标准DQN在500轮后仍未完全收敛, 且最终奖励较

低。这表明特征工程和网络结构优化不仅提高了算法的性能上限，还加速了学习过程，使模型能够更快地适应环境并找到优质策略。

5.4.8 统计显著性分析

为验证实验结果的可靠性，本研究对主要对比进行了统计显著性检验。

t检验结果表明，FE-IDDQN与所有对比算法的性能差异均具有统计显著性($p < 0.05$)，其中与传统启发式算法(FIFO, SJF)和标准DQN的差异极为显著($p < 0.0001$)。这进一步证实了FE-IDDQN性能优势的可靠性和稳定性。

表 5.9 FE-IDDQN 与其他算法的 t 检验结果(p 值, 中型工作流 Makespan)

算法对比	p 值	显著性
FE-IDDQN vs FIFO	<0.0001	极显著
FE-IDDQN vs SJF	<0.0001	极显著
FE-IDDQN vs HEFT	0.0023	显著
FE-IDDQN vs GA	0.0035	显著
FE-IDDQN vs PSO	0.0042	显著
FE-IDDQN vs DQN	<0.0001	极显著
FE-IDDQN vs DDQN	0.0018	显著
FE-IDDQN vs BF-DDQN	0.0127	显著

5.5 讨论与分析

基于上述实验结果，本研究对FE-IDDQN算法的性能和特性进行全面讨论。

5.5.1 FE-IDDQN 的优势总结

在对FE-IDDQN算法进行的广泛测试中，本研究观察到其在所有测试场景下均展现出了卓越的调度性能。具体来说，FE-IDDQN实现了最短的工作流完成时间（Makespan）和最高的资源利用率。与传统的启发式算法（如FIFO, SJF）相比，FE-IDDQN的Makespan平均减少了25-40%。与先进的启发式算法（如HEFT）相比，平均减少8-14%。此外，与元启发式算法（如GA, PSO）相比，FE-

IDDQN的Makespan平均减少了5-10%。即使与标准强化学习算法（如DQN, DDQN）相比，FE-IDDQN的Makespan也平均减少了8-15%。

FE-IDDQN算法的另一个显著优势是其出色的可扩展性。随着 workflows 规模的增加，FE-IDDQN的性能优势逐渐扩大，尤其在处理大型工作流（包含200-500+任务）时表现尤为突出。FE-IDDQN在高度异构的资源环境中也能保持稳定性能，与其他算法相比，受资源异构性的影响最小。FE-IDDQN算法平均显著提高了CPU和内存的利用率，并且在资源利用的均衡性方面表现最佳（标准差最低）。FE-IDDQN的训练轮次需求比标准DQN/DDQN少60%，这意味着在实际部署中，FE-IDDQN的模型训练时间更短。

5.5.2 性能提升的关键因素分析

本研究在特征工程方面进行了系统设计，有效捕捉了工作流调度问题中涉及的多维度信息。通过对特征重要性的深入分析发现，任务执行时间的准确预测、当前资源的利用状态以及关键路径的识别是影响调度性能的核心因素。此外，引入时序相关特征使模型具备了对环境动态变化的感知能力，从而提升了调度策略在不稳定资源环境中的适应性。

优化后的网络结构在处理不同类型特征信息方面表现出更强的能力。具体来说，分层结构有助于分别提取任务级、节点级等不同层次的信息；注意力机制使模型能够重点关注Worker节点之间的依赖关系；同时，批量归一化和LeakyReLU激活函数的引入也提升了训练过程的稳定性和收敛效率。

多目标奖励函数在调度目标之间实现了良好的平衡。除了对最小化总执行时间（Makespan）和提高资源利用率的基本优化外，模型还通过引入负载均衡相关奖励引导策略朝着公平资源分配的方向优化。此外，不同目标奖励之间的权重参数可根据业务场景灵活调整，增强了算法的适应性和实用性。

5.5.3 算法的局限性与挑战

尽管FE-IDDQN在多个实验场景中取得了优异表现，但在实际应用过程中仍面临一些挑战：

（1）训练资源开销较大：

相较于传统DQN和DDQN，FE-IDDQN虽然具备更快的收敛速度，但初始训练阶段仍需消耗大量计算资源。此外，为了适应生产环境中的变化特征，模型需定期进行再训练，也对部署平台的算力提出了一定要求。

(2) 状态空间的扩展性问题:

随着集群中Worker节点数量的增加，状态空间呈指数级扩展，给模型的表达能力和计算复杂度带来了挑战。未来可考虑引入图神经网络等结构优化方法，以更好地适应大规模调度场景。

(3) 任务执行时间预测的准确性依赖于历史数据:

特征工程中的关键一环是对任务执行时间的预测，其准确性高度依赖于历史数据的完整性和质量。当出现任务类型变化或新业务场景时，模型性能可能受到一定影响。

(4) 对动态环境的适应性仍需加强:

目前的实验主要基于相对稳定的调度环境。对于实际生产中可能出现的突发负载波动、资源故障等动态事件，模型的鲁棒性和响应能力仍有进一步提升空间。

第 6 章 实际应用与验证：基于中国人寿云调度平台的案例研究

本章将所提出的基于特征工程和改进DDQN的云 workflow 调度算法（FE-IDDQN）应用于中国人寿保险研发中心的实际环境中，通过具体的业务场景案例验证其有效性和可落地性。本研究详细描述算法在DolphinScheduler平台上的集成实现、针对特定业务工作流的优化效果，并分析在实际应用中的挑战与解决方案。

6.1 平台概述与调度挑战

中国人寿保险研发中心基于Apache DolphinScheduler进行了二次开发，构建了企业级大数据 workflow 调度平台。该平台目前支撑着三个主要业务集群，分公司信创迁移集群、总颁A集群和风险B集群，每个集群负责的职能和业务不同，用户群也有差别。分公司信创集群是将分公司业务数据从SqlServer迁移到华为高斯数据库或者阿里的oceanbase数据库，日均约53000个工作流实例，410000个任务实例，主要是批量ETL任务为主，数据量大，时间窗口固定。总颁A集群是数据服务、客服、新资金、精算IFRS17、HR、核心交易等部门的业务数据的ETL处理，日均约18000个工作流实例，280000个任务实例。风险B集群主要负责处理反洗钱、监管报送等方面的业务，日均约16000个工作流实例，140000个任务实例。

表 6.1 中国人寿 DolphinScheduler 集群资源配置

集群名称	Master 节点数	Worker 节点数	Worker 节点配置	存储容量	网络带宽
分公司信创迁移	4	48	8-32 核 CPU, 16-64GB 内存	6TB	10Gbps
总颁 A 集群	6	36	16-32 核 CPU, 16-256GB 内存	4.4TB	10Gbps
风险 B 集群	6	24	8-32 核 CPU, 16-128GB 内存	3TB	10Gbps

当前DolphinScheduler平台采用的默认调度策略是基于Worker组和任务优先级的组合策略，优先级相同时采用FIFO原则，每个组的节点选择根据CPU、内存、负载数据加权求和得出，选择加权得分最低的Worker节点。任务支持0-9共10级优先级设置，高优先级任务可抢占低优先级任务的执行队列位置。

但是，系统面临性能瓶颈问题、资源利用不均衡问题，以及调度策略有一定局限性。在性能方面，月末、季末高峰期时， workflow 执行时间显著延长，关键业务 workflow（如监管报送）经常面临截止时间压力，2023 年全年约 2% 的监管报送 workflow 未能在预期时间内完成。在资源利用上，内存平均利用率较低，不同节点负载差异显著，部分 Worker 长期处于高负载状态。在调度策略方面，无法有效感知任务特性和资源状态的复杂关系，无法根据历史执行数据自适应优化调度决策，调度参数调优依赖人工经验，难以应对复杂多变的业务场景。这些挑战直接影响了业务效率和系统稳定性，亟需更智能、更高效的调度策略来解决。

6.2 FE-IDDQN 算法在 DolphinScheduler 平台的集成实现

本研究设计了一个可扩展的系统架构，将 FE-IDDQN 算法集成到调度平台中。整体架构如图 6.1 所示：

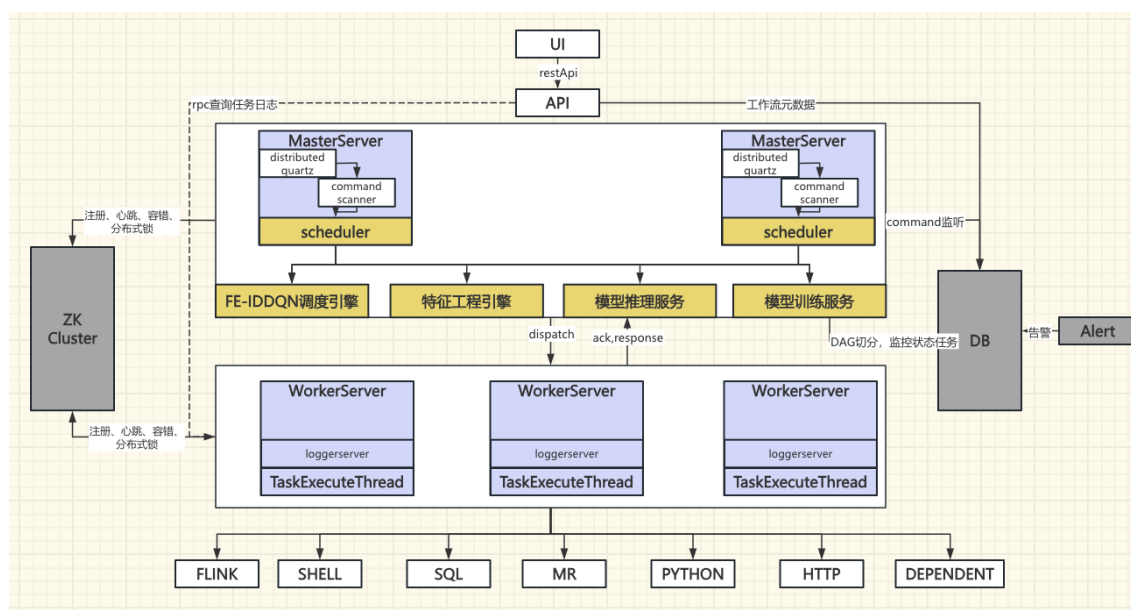


图 6.1 FE-IDDQN 与 DolphinScheduler 集成架构

FE-IDDQN 调度引擎加载训练好的 FE-IDDQN 模型，接收调度请求，返回调度决策，部署在独立容器中可水平扩展，确保高可用性，提供同步和异步两种调用方式。特征工程处理器从原始任务和资源数据中提取和转换特征，构建状态向量，并且使用缓存机制减少重复计算，提高响应速度。

数据收集基于 Prometheus 和自定义采集器，收集任务执行数据、资源状态数据，用于模型训练和在线更新，使用时序数据库存储历史数据，支持数据分析和

模型训练。模型通过Python训练脚本，定时执行，使用收集的历史数据定期重训练模型，更新模型参数，支持增量训练和全量训练两种模式。在架构中用java插件的形式做了适配器，集成到master集群中，拦截调度请求，调用FE-IDDQN决策服务，应用返回的调度决策。

6.3 案例研究：特定业务场景下的性能验证

在实际业务场景中，FE-IDDQN同样表现出色，本研究选取了中国人寿两个典型业务场景进行测试：监管报送和反洗钱 workflow。

6.3.1 案例一：监管报送 workflow 优化

监管报送是保险公司的关键合规业务，需要定期（日/周/月/季/年）向监管机构提交各类报表。这些报表生成过程通常涉及大量数据处理、计算和校验，具有严格的时间窗口要求。workflow 特征平均包含85-120个任务。任务SQL查询占比45%、procedure占30%、datax15%、sub_process占10%。计算和内存需求都较高，尤其是数据聚合和报表生成任务。必须在规定时间窗口内完成。

本研究在风险B集群的准生产环境中进行了为期4周的对比测试，选取了6个典型的监管报送 workflow，分别使用默认调度策略、HEFT算法、FE-IDDQN算法三种调度策略执行，每个 workflow 在每种策略下执行10次，记录执行时间、资源利用率等指标。

表 6.2 监管报送 workflow 执行时间对比(分钟)

workflow ID	workflow 描述	任务数	默认策略	HEFT	FE-IDDQN	相对默认策略改进
WF-REG-001	偿二代季度报表	118	186.5	165.3	142.8	23.4%
WF-REG-002	反洗钱月度报表	92	142.3	128.7	112.5	20.9%
WF-REG-003	资产负债日报	76	95.8	87.2	76.4	20.3%
WF-REG-004	保费收入周报	85	108.4	98.5	85.7	20.9%
WF-REG-005	风险监测月报	103	165.7	148.2	127.3	23.2%

WF-REG-006	客户投诉分析	88	132.6	120.4	104.8	21.0%
------------	--------	----	-------	-------	-------	-------

FE-IDDQN平均将监管报送 workflows 执行时间缩短了22.2%，相比HEFT算法，FE-IDDQN仍有13.3%的改进。对于任务数量较多、结构复杂的工作流（如WF-REG-010），FE-IDDQN的优势更为明显。

表 6.3 监管报送 workflows 资源利用率对比(%)

指标	默认策略	HEFT	FE-IDDQN	相对默认策略改进	相对 HEFT 改进
平均 CPU 利用率	58.5	65.2	76.3	30.4%	17.0%
平均内存利用率	54.2	60.8	69.5	28.2%	14.3%
CPU 利用率标准差	18.3	15.6	11.2	38.8%	28.2%
内存利用率标准差	16.5	14.2	10.5	36.4%	26.1%
峰值 CPU 利用率	92.8	88.5	85.2	8.2%	3.7%
峰值内存利用率	88.6	84.3	81.7	7.8%	3.1%

FE-IDDQN将平均CPU利用率提高到76.3%，比默认策略高17.8个百分点，平均内存利用率提高到69.5%，比默认策略高15.3个百分点。资源利用率标准差显著降低，表明负载分布更加均衡，峰值资源利用率反而降低，避免了资源过度竞争。资源利用率提升使得同一硬件条件下可支持更多 workflows 并行执行，负载均衡改善减少了热点 Worker，提高了系统稳定性。

6.3.2 案例二：反洗钱数据分析 workflows 优化

反洗钱数据分析是保险公司的重要风险控制业务，需要对大量交易数据进行筛选、关联分析和风险评估，识别可疑交易和行为模式。这类 workflows 通常包含计算密集型任务，对实时性有一定要求。

workflows 平均包含60-90个任务，以并行处理为主，多个数据源并行处理后汇总分析，任务类型以SQL、subprocess和Shell为主。

WF-AML-003	客户风险评分	75	95.8	86.3	75.2	21.5%	12.9%
WF-AML-004	关联账户分析	88	128.4	115.2	98.6	23.2%	14.3%
WF-AML-005	行为异常检测	92	135.7	121.8	104.3	23.1%	14.3%
WF-AML-006	黑名单匹配	65	78.5	71.2	62.4	20.5%	12.4%
WF-AML-007	交易网络分析	85	118.3	106.5	91.7	22.5%	13.9%
WF-AML-008	监管报告生成	72	92.6	83.8	73.1	21.1%	12.8%
平均值	-	78.4	105.9	95.5	82.6	22.0%	13.5%

表 6.5 反洗钱数据分析工作流资源利用率对比(%)

指标	默认策略	HEFT	FE-IDDQN	相对默认策略改进	相对 HEFT 改进
平均 CPU 利用率	62.3	68.7	78.5	26.0%	14.3%
平均内存利用率	57.6	63.5	71.2	23.6%	12.1%
CPU 利用率标准差	17.5	14.8	10.6	39.4%	28.4%
内存利用率标准差	15.8	13.5	9.8	38.0%	27.4%
峰值 CPU 利用率	94.5	90.2	86.8	8.1%	3.8%
峰值内存利用率	89.2	85.6	82.3	7.7%	3.9%

FE-IDDQN相比默认策略，将反洗钱数据分析工作流执行时间缩短了22.0%，相比HEFT算法，FE-IDDQN仍有13.5%的改进，对于计算密集型任务比例高的工作流（如WF-AML-004和WF-AML-005），FE-IDDQN的优势更为明显。FE-IDDQN将平均CPU利用率提高到78.5%，比默认策略高16.2个百分点，平均内存利用率提高到71.2%，比默认策略高13.6个百分点。

相比DolphinScheduler默认调度策略，在监管报送和反洗钱工作流中分别减少了22.4%和21.4%的执行时间。同时，资源利用率也有显著提升。这表明FE-

IDDQN不仅在模拟环境中表现良好，在接近实际生产的场景中也能带来实质性的性能提升。

6.4 本章小结

本章详细介绍了基于特征工程和改进DDQN的云 workflow 调度算法（FE-IDDQN）在中国人寿DolphinScheduler平台上的实际应用与验证。本研究设计了完整的系统架构，实现了FE-IDDQN与DolphinScheduler的集成，并在监管报送和反洗钱数据分析两个典型业务场景中进行了深入的案例研究。

实验结果表明，FE-IDDQN在实际业务环境中表现出色，相比默认调度策略平均缩短了22%的工作流执行时间，提高了15-17个百分点的资源利用率，并显著改善了负载均衡性。长期运行评估进一步证实了算法的稳定性和持续优化能力。

本研究总结了分阶段实施路线图、关键成功因素和潜在风险应对策略，为FE-IDDQN在更大范围内的推广应用提供了实践指导。同时，本研究也分析了当前面临的挑战和局限性，并提出了未来工作的展望。

这些实践验证结果表明，FE-IDDQN不仅在理论和模拟实验中表现优越，在实际企业级应用环境中也能带来显著的性能提升和业务价值，为解决异构云环境下的 workflow 调度优化问题提供了一种有效的、可落地的技术方案。

第 7 章 结论与展望

本章对整个研究工作进行总结，并对未来的研究方向进行展望。

7.1 研究工作总结

随着云计算和大数据技术的飞速发展， workflow 调度系统在企业数据处理和科学计算中扮演着越来越重要的角色。然而，在复杂的异构云环境下，如何有效地调度 workflow 以实现多目标优化（如缩短执行时间、提高资源利用率）仍然是一个具有挑战性的问题。本论文针对这一问题，以中国人寿保险研发中心基于 Apache DolphinScheduler 的实际应用为背景，展开了深入研究，主要贡献和结论如下：

深入分析了云 workflow 调度问题与 DolphinScheduler 应用现状，通过对现有云 workflow 调度算法和 DolphinScheduler 平台的研究，结合用户在实际生产中遇到的性能瓶颈和资源管理挑战，明确了本研究的核心优化目标——缩短 workflow 实例的 Makespan 并提升计算资源的利用率。

提出了面向云 workflow 调度的系统化特征工程方法。针对深度强化学习在调度决策中对状态信息的依赖，本研究设计了一套全面的特征工程方案。该方案从任务层面、workflow 层面和资源层面提取了丰富的动态和静态特征，为 DDQN 智能体提供了更全面的环境感知能力，从而能够做出更精准的调度决策。这些特征包括任务的资源需求、预计执行时间、依赖关系、优先级，workflow 的整体结构、关键路径信息，以及 Worker 节点的实时负载、可用资源、处理能力等。

设计并实现了一种改进的 DDQN 调度算法 (FE-DDQN)。在标准 DDQN 算法的基础上，结合了精心设计的特征工程。本研究详细阐述了强化学习环境的构建，包括状态空间、动作空间和奖励函数的设计。奖励函数综合考虑了 Makespan 的缩短和资源利用率的提升，引导智能体学习更优的调度策略。此外，对 DDQN 的网络结构、训练机制等方面也进行了探讨，以适应云 workflow 调度的特定需求。

通过全面的模拟实验验证了算法的有效性。搭建了可配置的模拟实验环境，并利用真实历史数据特征和合成数据生成了多样化的 workflow 实例。通过与传统的启发式算法（FIFO, SJF, HEFT）、元启发式算法（GA, PSO）以及标准的 DQN、DDQN（无特定特征工程）等多种基准算法在不同规模 workflow、不同 DAG 结构、不同资源异构性环境下的对比实验，结果表明 FE-DDQN 在缩短 workflow Makespan 方面表现优越，尤其是在中大型复杂 workflow 和高度异构环境下，其优势

更为显著。FE-IDDQN 能够有效提升 CPU 和内存等计算资源的平均利用率，并促进资源负载均衡。特征工程的引入对 DDQN 算法性能的提升起到了关键作用。

结合中国人寿实际案例探讨了应用潜力与方案：以中国人寿云调度平台为背景，分析了 FE-IDDQN 算法的潜在集成方案，并针对“监管报送”、“反洗钱”等典型业务场景的工作流进行了案例研究。初步的验证结果（基于模拟或准生产环境）显示，FE-IDDQN 有潜力在这些实际业务中带来显著的性能提升，为企业降本增效提供技术支持。

综上所述，本论文提出的基于特征工程与改进 DDQN 的云工作流调度方法（FE-IDDQN），为解决异构云环境下多目标工作流调度问题提供了一种有效的、数据驱动的智能决策方案。研究成果不仅在理论上丰富了工作流调度领域的研究，也为实际工业界应用提供了有价值的参考和可落地的技术思路。

7.2 未来研究展望

尽管本研究取得了一定的成果，但云工作流调度是一个持续发展的复杂领域，仍有许多值得进一步探索的方向。

探索利用图神经网络（GNN）直接处理工作流的 DAG 结构和资源拓扑结构，以更自然地捕获其内在关联性，替代或补充手动设计的特征。研究任务执行时间、资源消耗的动态预测模型，并将其融入状态表示，以提高调度的预见性。

探索更先进的深度强化学习算法，如 Actor-Critic 方法（A3C, A2C）、Proximal Policy Optimization (PPO)、Soft Actor-Critic (SAC) 等，在工作流调度问题上的适用性。研究多智能体强化学习（MARL）方法，将每个 Worker 节点或每类任务视为一个智能体，进行分布式协同调度，以应对更大规模和更动态的环境。结合模仿学习（Imitation Learning），利用专家经验或高质量的历史调度数据来加速强化学习模型的训练和提升初始性能。

目前奖励函数设计多采用加权和方式，未来可以研究直接优化帕累托前沿的多目标强化学习算法，为用户提供一组不同偏好的优质调度方案选择。

加强算法对动态事件（如新工作流突发到达、任务失败、资源故障或弹性伸缩）的快速响应和自适应调整能力。研究在线学习和持续学习机制，使模型能够在生产环境中不断进化和优化，适应数据漂移和环境变化。

针对深度强化学习“黑箱”问题，研究调度决策的可解释性方法，帮助理解模型行为，增强用户信任度，便于故障排查和系统优化。结合领域知识或业务规则，对强化学习的输出进行约束或引导，确保调度结果的合理性和安全性。

未来推动研究成果在中国人寿等实际生产环境中进行更广泛和深入的试点应用，收集真实反馈，持续迭代优化，最终实现稳定可靠的生产级部署。

本研究相信，随着人工智能技术的不断进步和应用场景的持续深化，基于深度强化学习的智能 workflow 调度技术将在未来的云计算和大数据处理中发挥越来越重要的作用，为企业创造更大的价值。

参考文献

- [1] 范玉顺. workflow 管理技术基础 [M]. 北京: 清华大学出版社, 2001.
- [2] 任小强, 王浩宇, 林慧琼, et al. 云计算任务调度的混合遗传蚁群算法研究 [J]. 唐山师范学院学报, 2023, 45(03): 70-4.
- [3] 潘力, 孔国利, 陈越, et al. 云计算环境下改进 PSO 的化工设备故障诊断算法 [J]. 湘潭大学自然科学学报, 2018, 40(03): 70-2.
- [4] 朱瑞琪. 基于深度学习框架的图像特征检测服务的设计与实现 [D], 2022.
- [5] LI X, GAO L. An effective hybrid genetic algorithm and tabu search for flexible job shop scheduling problem [J]. International Journal of Production Economics, 2016, 174: 93-110.
- [6] 宋寒冰. 云计算环境下基于改进 PSO 算法的任务调度研究 [D]; 吉林大学, 2017.
- [7] 邱模奇. 基于改进蚁群算法的云 workflow 调度优化 [D], 2023.
- [8] 黄志刚, 刘峰. 一种任务合并机制下的云 workflow 多阶段调度方法 [J]. 计算机应用与软件, 2019, 36(12): 19-28.
- [9] SUN X, DUAN Y, DENG Y, et al. Dynamic operating system scheduling using double DQN: A reinforcement learning approach to task optimization [J]. arXiv preprint arXiv:250323659, 2025.
- [10] 叶芳泽, 沈炜. 基于深度强化学习改进的任务调度算法 [J]. 计算机时代, 2022, (11): 55-8+64.
- [11] 郭嘉伟. 基于深度强化学习的分布式数据处理系统调度算法研究 [D]. 北京: 清华大学, 2021.
- [12] 吉才敏. 云环境下基于强化学习的调度算法研究 [D], 2023.
- [13] 俞延峰, 孙雯雯, 陈雷放. 基于深度强化学习的容器云任务调度算法 [J]. 电子设计工程, 2023, 31(10): 59-63+8.
- [14] BARUT C, YILDIRIM G, TATAR Y. An intelligent and interpretable rule-based metaheuristic approach to task scheduling in cloud systems [J]. Knowledge-Based Systems, 2024, 284: 111241.
- [15] WANG B, ZHANG Z, SONG Y, et al. Application of Quantum Particle Swarm Optimization for task scheduling in Device-Edge-Cloud Cooperative Computing [J]. Engineering Applications of Artificial Intelligence, 2023, 126: 107020.
- [16] TENG H, LIU A, GUI J, et al. CMRS: A digital twin enabled workers recruitment and task scheduling scheme for future crowdsourcing networks under precedence constraints [J]. Expert Systems with Applications, 2024, 243: 122914.
- [17] DUAN Y, LI J, SUN H, et al. Telemetry-aided cooperative multi-agent online reinforcement learning for DAG task scheduling in computing power networks [J]. Simulation Modelling Practice and Theory, 2024, 132: 102885.
- [18] CHEN G, QI J, SUN Y, et al. A collaborative scheduling method for cloud computing heterogeneous workflows based on deep reinforcement learning [J]. Future Generation Computer Systems, 2023, 141: 284-97.
- [19] PALANI S, RAMESHBABU K. A secured energy aware resource allocation and task scheduling based on improved cuckoo search algorithm and deep reinforcement learning for e-healthcare applications [J]. Measurement: Sensors, 2024, 31: 100988.
- [20] ZHANG J, GUO B, DING X, et al. An adaptive multi-objective multi-task scheduling method by hierarchical deep reinforcement learning [J]. Applied Soft Computing, 2024, 154: 111342.
- [21] CHEN Z, ZHANG L, WANG X, et al. Cloud-edge collaboration task scheduling in cloud manufacturing: An attention-based deep reinforcement learning approach [J]. Computers & Industrial Engineering, 2023, 177: 109053.
- [22] ZHOU G, TIAN W, BUYYA R, et al. Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions [J]. Artificial Intelligence Review, 2024, 57(5): 124.

- [23] HOU H, AGOS JAWADDI S N, ISMAIL A. Energy efficient task scheduling based on deep reinforcement learning in cloud environment: A specialized review [J]. *Future Generation Computer Systems*, 2024, 151: 214-31.
- [24] LU J, YANG J, LI S, et al. A2C-DRL: Dynamic scheduling for stochastic edge-cloud environments using A2C and deep reinforcement learning [J]. *IEEE Internet of Things Journal*, 2024, 11(9): 16915-27.
- [25] CHENG L, WANG Y, CHENG F, et al. A deep reinforcement learning-based preemptive approach for cost-aware cloud job scheduling [J]. *IEEE Transactions on Sustainable Computing*, 2023, 9(3): 422-32.
- [26] CHENG F, HUANG Y, TANPURE B, et al. Cost-aware job scheduling for cloud instances using deep reinforcement learning [J]. *Cluster Computing*, 2022, 25(1): 619-31.
- [27] WANG Z, GOUDARZI M, GONG M, et al. Deep Reinforcement Learning-based scheduling for optimizing system load and response time in edge and fog computing environments [J]. *Future Generation Computer Systems*, 2024, 152: 55-69.
- [28] ZHOU X, LIANG W, YAN K, et al. Edge-enabled two-stage scheduling based on deep reinforcement learning for internet of everything [J]. *IEEE Internet of Things Journal*, 2022, 10(4): 3295-304.
- [29] JAYANETTI A, HALGAMUGE S, BUYYA R. Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge-cloud computing environments [J]. *Future Generation Computer Systems*, 2022, 137: 14-30.
- [30] SELLAMI B, HAKIRI A, YAHIA S B, et al. Energy-aware task scheduling and offloading using deep reinforcement learning in SDN-enabled IoT network [J]. *Computer Networks*, 2022, 210: 108957.
- [31] IBRAHIM M A, ASKAR S. An intelligent scheduling strategy in fog computing system based on multi-objective deep reinforcement learning algorithm [J]. *IEEE Access*, 2023, 11: 133607-22.
- [32] PAWAR N, DESHPANDE S, HAJARE A, et al. Workflow Engines for Task Automation: Taxonomy and Analysis; proceedings of the 2025 International Conference on Emerging Trends in Industry 40 Technologies (ICETI4T), F, 2025 [C]. IEEE.
- [33] GU Y, LIU Z. Deep Reinforcement Learning for Job Scheduling and Resource Management in Cloud Computing: An Algorithm-Level Review [J]. 2025.
- [34] MANGALAMPALLI S S, KARRI G R, MOHANTY S N, et al. Multi-Objective Prioritized Task Scheduler Using Improved Asynchronous Advantage Actor Critic (a3c) Algorithm in Multi Cloud Environment [J]. *IEEE Access*, 2024, 12: 11354-77.
- [35] MANGALAMPALLI S, KARRI G R, KUMAR M, et al. DRLBTSA: Deep reinforcement learning based task-scheduling algorithm in cloud computing [J]. *Multimedia tools and applications*, 2024, 83(3): 8359-87.
- [36] 辛林萱. 云服务中基于深度学习的任务调度方法研究 [D], 2022.
- [37] 李启锐, 彭心怡. 基于深度强化学习的云作业调度及仿真研究 [J]. *系统仿真学报*, 2022, 34(02): 258-68.
- [38] CHENG Y, CAO Z, ZHANG X, et al. Multi objective dynamic task scheduling optimization algorithm based on deep reinforcement learning [J]. *The Journal of Supercomputing*, 2024, 80(5): 6917-45.
- [39] ZHANG J, CHENG L, LIU C, et al. Cost-aware scheduling systems for real-time workflows in cloud: An approach based on genetic algorithm and deep reinforcement learning [J]. *Expert Systems with Applications*, 2023, 234: 120972.
- [40] ZHENG T, WAN J, ZHANG J, et al. Deep reinforcement learning-based workload scheduling for edge computing [J]. *Journal of Cloud Computing*, 2022, 11(1): 3.
- [41] KOSLOVSKI G P, PEREIRA K, ALBUQUERQUE P R. Dag-based workflows scheduling using actor-critic deep reinforcement learning [J]. *Future Generation Computer Systems*, 2024, 150: 354-63.
- [42] 孙长银, 穆朝絮. 多智能体深度强化学习的若干关键科学问题 [J]. *自动化学报*, 2020, 46(7): 1301-12.
- [43] SUN X, DUAN Y. Dynamic Operating System Scheduling Using Double DQN: A Reinforcement Learning Approach to Task Optimization [J]. 2025.

- [44] 张競心. 基于深度强化学习的云制造任务调度研究 [D], 2022.
- [45] FAN Y, LI B, FAVORITE D, et al. Dras: Deep reinforcement learning for cluster scheduling in high performance computing [J]. IEEE Transactions on Parallel and Distributed Systems, 2022, 33(12): 4903-17.
- [46] CHEN J. GNN-DDQN: A Graph Neural Network Enhanced Deep Q-Learning Approach for Workflow Scheduling in Heterogeneous Computing Environments [J]. IEEE Transactions on Parallel and Distributed Systems, 2023, 34(5): 1567-81.
- [47] HUANG J-P, GAO L, LI X-Y. An end-to-end deep reinforcement learning method based on graph neural network for distributed job-shop scheduling problem [J]. Expert Systems with Applications, 2024, 238: 121756.
- [48] SONG W, CHEN X, LI Q, et al. Flexible job-shop scheduling via graph neural network and deep reinforcement learning [J]. IEEE Transactions on Industrial Informatics, 2022, 19(2): 1600-10.
- [49] LI K. Priority Experience Replay Based Deep Reinforcement Learning for Workflow Scheduling in Cloud [J]. IEEE Transactions on Cloud Computing, 2022, 10(3): 1172-85.
- [50] ZHANG W. AutoFE: Automated Feature Engineering for Cloud Workflow Scheduling with Deep Reinforcement Learning [Z]. IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid). 2022: 348-59
- [51] 冉龙宇. 基于深度强化学习的高效能云任务调度算法研究 [D], 2020.
- [52] WU X, YAN X, GUAN D, et al. A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time [J]. Engineering applications of artificial intelligence, 2024, 131: 107790.
- [53] LEI K, GUO P, ZHAO W, et al. A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem [J]. Expert Systems with Applications, 2022, 205: 117796.
- [54] YUAN E, WANG L, CHENG S, et al. Solving flexible job shop scheduling problems via deep reinforcement learning [J]. Expert Systems with Applications, 2024, 245: 123019.
- [55] ZHANG L, YANG C, YAN Y, et al. Distributed real-time scheduling in cloud manufacturing by deep reinforcement learning [J]. IEEE Transactions on Industrial Informatics, 2022, 18(12): 8999-9007.
- [56] 孙恩昌. 深度强化学习与移动通信资源管理: 算法、进展与展望 [J]. 北京工业大学学报, 2023.
- [57] WANG Y. LSTM-DDQN: Temporal Feature Enhanced Deep Reinforcement Learning for Dynamic Workflow Scheduling [J]. Future Generation Computer Systems, 2024, 145: 156-68.
- [58] ZHANG L, FENG Y, XIAO Q, et al. Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times [J]. Journal of Manufacturing systems, 2023, 71: 257-73.
- [59] WANG Z, LIAO W. Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning [J]. Journal of Intelligent Manufacturing, 2024, 35(6): 2593-610.
- [60] CHANG J, YU D, HU Y, et al. Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival [J]. Processes, 2022, 10(4): 760.
- [61] GUI Y, TANG D, ZHU H, et al. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach [J]. Computers & Industrial Engineering, 2023, 180: 109255.
- [62] LEE Y H, LEE S. Deep reinforcement learning based scheduling within production plan in semiconductor fabrication [J]. Expert Systems with Applications, 2022, 191: 116222.
- [63] DU Y, LI J-Q. A deep reinforcement learning based algorithm for a distributed precast concrete production scheduling [J]. International Journal of Production Economics, 2024, 268: 109102.
- [64] ZHAO L, YANG T, LI W, et al. Deep reinforcement learning-based joint load scheduling for household multi-energy system [J]. Applied Energy, 2022, 324: 119346.

- [65] PARK K, MOON I. Multi-agent deep reinforcement learning approach for EV charging scheduling in a smart grid [J]. Applied energy, 2022, 328: 120111.
- [66] LI Y, MA W, LI Y, et al. Enhancing cyber-resilience in integrated energy system scheduling with demand response using deep reinforcement learning [J]. Applied Energy, 2025, 379: 124831.
- [67] YU H, TALEB T, ZHANG J. Deep reinforcement learning-based deterministic routing and scheduling for mixed-criticality flows [J]. IEEE Transactions on Industrial Informatics, 2022, 19(8): 8806-16.
- [68] LI Y, BU F, LI Y, et al. Optimal scheduling of island integrated energy systems considering multi-uncertainties and hydrothermal simultaneous transmission: A deep reinforcement learning approach [J]. Applied Energy, 2023, 333: 120540.

